electrical-engineering: the Electrical Engineering program — user guide

electrical-engineering-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Electrical Engineering covers DC and AC circuits, transients, phasors and power, operational amplifiers, diodes and transistors, digital logic, and control systems (transfer functions, stability, Bode). This guide is about `electrical-engineering`, the model as a self-contained program for Windows and Linux: 64 capabilities in 7 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/electrical-engineering-ai999x/); the Python package (`pip install electrical-engineering-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Electrical Engineering	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-electrical-engineering-1.4.0-windows-x86_64.zip
Desktop app: Electrical Engineering	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-electrical-engineering-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Electrical Engineering in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-electrical-engineering-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-electrical-engineering.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-electrical-engineering-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-electrical-engineering-1.4.0-linux-x86_64.tar.gz
./ai999x-electrical-engineering-1.4.0-linux-x86_64/ai999x-electrical-engineering
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `electrical-engineering` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
electrical-engineering --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";$D", "User")
electrical-engineering --version
```

In your browser

Open https://www.ai999x.com/models/electrical-engineering-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install electrical-engineering-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ electrical-engineering --version
electrical-engineering-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ electrical-engineering ask "Find the Thevenin equivalent between nodes 2 and 0 of the circuit: V1
1 0 12; R1 1 2 1000; R2 2 0 2000."
Read as: Thevenin equivalent of a DC circuit between two nodes
Vth = 8 V
Rth = 666.6666667 ohm
  by: Thevenin equivalent of a DC circuit between two nodes  (open-circuit voltage by modified
nodal analysis; resistance by a 1 A test current with the independent sources set to zero)
  netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000.
  node_a = 2
  node_b = 0
  assumes: both nodes in the circuit; no ideal voltage source directly across the terminals; a
unique solution (no floating node)
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ electrical-engineering search "thevenin equivalent of a dc circuit between two nodes"
thevenin_equivalent                  Thevenin equivalent of a DC circuit between two nodes  (DC
circuits / Thevenin and Norton equivalents)
norton_equivalent                    Norton equivalent of a DC circuit between two nodes  (DC
circuits / Thevenin and Norton equivalents)
dc_node_voltages                     Node voltages of a DC resistive circuit from its netlist  (DC
circuits / mesh and nodal analysis)
boolean_equivalence                  Equivalence of two Boolean expressions  (digital logic /
Boolean algebra)
nodal_voltages                       Node voltages from the nodal equations (Kirchhoff's current
law)  (DC circuits / mesh and nodal analysis)
max_power_transfer                   Maximum power transfer  (DC circuits / Thevenin and Norton
equivalents)
series_resistance                    Equivalent resistance of resistors in series  (DC circuits /
series and parallel combinations)
parallel_resistance                  Equivalent resistance of resistors in parallel  (DC circuits /
series and parallel combinations)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ electrical-engineering list
adc_conversion                           Analog-to-digital conversion: output code and resolution
base_conversion                          Number base conversion (binary, octal, decimal,
hexadecimal)
bjt_divider_bias                         DC operating point of a voltage-divider biased BJT
bode_point                               Frequency response (Bode magnitude and phase) at one
frequency
boolean_equivalence                      Equivalence of two Boolean expressions
boolean_evaluate                         Evaluate a Boolean expression
closed_loop_poles                        Closed-loop poles with unity negative feedback
complex_power                            Complex, real, reactive and apparent power from voltage
and current phasors
… 56 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ electrical-engineering info thevenin_equivalent
Thevenin equivalent of a DC circuit between two nodes
  open-circuit voltage by modified nodal analysis; resistance by a 1 A test current with the
independent sources set to zero
  in   netlist  elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node->
<volts>, I<name> <node> <node> <amperes>; node 0 is ground (text) []
  in   node_a   the terminal node (the + side of the equivalent) (text) []
  in   node_b   the reference terminal node (text) []
  out  Vth      Thevenin (open-circuit) voltage, node a relative to node b [V]
  out  Rth      Thevenin resistance [ohm]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ electrical-engineering solve thevenin_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2
1000; R2 2 0 2000."
Vth = 8 V
Rth = 666.6666667 ohm
  by: Thevenin equivalent of a DC circuit between two nodes  (open-circuit voltage by modified
nodal analysis; resistance by a 1 A test current with the independent sources set to zero)
  node_a = 2
  netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000.
  node_b = 0
  assumes: both nodes in the circuit; no ideal voltage source directly across the terminals; a
unique solution (no floating node)
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ electrical-engineering solve thevenin_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2
1000; R2 2 0 2000." --json
{"outcome":"SOLVED","message":"","value":8.0,"unit":"V","solved_for":"Vth","values":
[["Vth",8.0,"V"],["Rth",666.6666666666666,"ohm"]],"target":"Vth","phrase":"Thevenin (open-circuit)
voltage, node a relative to node b","assumptions":["both nodes in the circuit","no ideal voltage
source directly across the terminals","a unique solution (no floating node)"],"name":"Thevenin
equivalent of a DC circuit between two nodes","equation":"open-circuit voltage by modified nodal
analysis; resistance by a 1 A test current with the independent sources set to zero"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ electrical-engineering solve thevenin_equivalent node_a=2 node_b=0
NEEDS_INPUT: thevenin_equivalent needs netlist (elements separated by ';': R<name> <node> <node>
<ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes>; node 0 is ground).
Example: solve('thevenin_equivalent', netlist='V1 1 0 12; R1 1 2 1000; R2 2 0 2000', node_a='2',
node_b='0').
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ electrical-engineering figures thevenin_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2
1000; R2 2 0 2000." --out .
[
  {
    "slug": "thevenin-equivalent-schematic",
    "title": "Thévenin equivalent between the two nodes",
    "caption": "The Thévenin equivalent computed for this circuit seen between node 2 (a) and node
0 (b): a source Vth = 8 V in series with Rth = 666.7 Ω. Your inputs: node_a = 2, node_b = 0,
netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-schematic.svg"
  },
  {
    "slug": "thevenin-equivalent-live-1",
    "title": "Thevenin (open-circuit) voltage, node a relat... as the number 0 in node_b (0)
varies",
    "caption": "Thevenin (open-circuit) voltage, node a relat... as the number in node_b runs from
0 to 1, every other input as you entered it (2 solutions). Marked: your the number in node_b = 0
gives Vth = 8. Your inputs: node_a = 2, node_b = 0, netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-live-1.svg"
  },
  {
    "slug": "thevenin-equivalent-live-2",
    "title": "Thevenin (open-circuit) voltage, node a relat... as the number 1 in netlist (V1 1 0
12; R1 1 2 100...) varies",
    "caption": "Thevenin (open-circuit) voltage, node a relat... as the number in netlist runs from
1 to 5, every other input as you entered it (4 solutions). Values in this range where the inputs
admit no answer (1 of 5) are left out. Marked: your the number in netlist = 1 gives Vth = 8. Your
inputs: node_a = 2, node_b = 0, netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-live-2.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | electrical-engineering batch

`cases.jsonl`:

```
{"key": "thevenin_equivalent", "inputs": {"node_a": "2", "node_b": "0", "netlist": "V1 1 0 12; R1 1
2 1000; R2 2 0 2000."}}
{"key": "gain_margin", "inputs": {"num": "[10]", "den": "[1, 6, 5, 0]"}}
```

```
$ electrical-engineering batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":8.0,"unit":"V","solved_for":"Vth","values":
[["Vth",8.0,"V"],["Rth",666.6666666666666,"ohm"]],"target":"Vth","phrase":"Thevenin (open-circuit)
voltage, node a relative to node b","assumptions":["both nodes in the circuit","no ideal voltage
source directly across the terminals","a unique solution (no floating node)"],"name":"Thevenin
equivalent of a DC circuit between two nodes","equation":"open-circuit voltage by modified nodal
analysis; resistance by a 1 A test current with the independent sources set to zero"}
{"outcome":"SOLVED","message":"","value":3.0,"unit":"","solved_for":"gain_margin","values":
[["gain_margin",3.0,""],["gain_margin_db",9.542425094393248,""],
["phase_crossover",2.23606797749979,"rad/s"]],"target":"gain_margin","phrase":"gain margin as a
ratio","assumptions":["a proper G(s)","exactly one phase crossover with G negative
real"],"name":"Gain margin and phase-crossover frequency","equation":"the phase crossover as the
positive real root of Im(N(jw) conj D(jw)) with Re G < 0 (Aberth iteration, Newton-polished)"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `electrical-engineering info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]"` or `"v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`electrical-engineering solve ... --json` prints one answer as one line of JSON. `electrical-engineering batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | electrical-engineering batch`. `electrical-engineering catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["electrical-engineering", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`electrical-engineering figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ electrical-engineering figures thevenin_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2
1000; R2 2 0 2000." --out .
[
  {
    "slug": "thevenin-equivalent-schematic",
    "title": "Thévenin equivalent between the two nodes",
    "caption": "The Thévenin equivalent computed for this circuit seen between node 2 (a) and node
0 (b): a source Vth = 8 V in series with Rth = 666.7 Ω. Your inputs: node_a = 2, node_b = 0,
netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-schematic.svg"
  },
  {
    "slug": "thevenin-equivalent-live-1",
    "title": "Thevenin (open-circuit) voltage, node a relat... as the number 0 in node_b (0)
varies",
    "caption": "Thevenin (open-circuit) voltage, node a relat... as the number in node_b runs from
0 to 1, every other input as you entered it (2 solutions). Marked: your the number in node_b = 0
gives Vth = 8. Your inputs: node_a = 2, node_b = 0, netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-live-1.svg"
  },
  {
    "slug": "thevenin-equivalent-live-2",
    "title": "Thevenin (open-circuit) voltage, node a relat... as the number 1 in netlist (V1 1 0
12; R1 1 2 100...) varies",
    "caption": "Thevenin (open-circuit) voltage, node a relat... as the number in netlist runs from
1 to 5, every other input as you entered it (4 solutions). Values in this range where the inputs
admit no answer (1 of 5) are left out. Marked: your the number in netlist = 1 gives Vth = 8. Your
inputs: node_a = 2, node_b = 0, netlist = V1 1 0 12; R1 1 2 1000; R....",
    "file": ".\\thevenin-equivalent-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `electrical_engineering_ai999x.wasm` (https://www.ai999x.com/compute/rust/electrical_engineering_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("electrical_engineering_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `electrical-engineering info <id>` shows the same inputs and outputs in the terminal.

AC circuits (14)

Balanced three-phase load (wye or delta) — `three_phase_load`

Phase voltage, phase and line currents, real, reactive and apparent power and the power factor of a balanced load with impedance R + jX per phase.

$$P = \sqrt{3}\, V_L I_L \cos\phi$$

symbol		meaning	SI unit
`VL`	in	rms line-to-line voltage	V
`R`	in	resistance per phase	ohm
`X`	in	reactance per phase (positive inductive, negative capacitive)	ohm
`connection`	in	load connection	—
`phase_voltage`	out	voltage across each phase of the load	V
`phase_current`	out	current in each phase of the load	A
`line_current`	out	current in each line	A
`P`	out	total real power in W	W
`Q`	out	total reactive power in var	W
`S`	out	total apparent power in VA	W
`power_factor`	out	power factor R/|Z|	—

Applies when: balanced load and supply; R >= 0.

```
$ electrical-engineering solve three_phase_load connection=wye "VL=400 V" "R=8 ohm" "X=6 ohm"
phase_voltage = 230.9401077 V
phase_current = 23.09401077 A
line_current = 23.09401077 A
P = 12800 W
Q = 9600 W
S = 16000 W
power_factor = 0.8
  by: Balanced three-phase load (wye or delta)  (wye: V_ph = V_L/sqrt(3), I_L = I_ph; delta: V_ph =
V_L, I_L = sqrt(3) I_ph; P = 3 I_ph^2 R)
  VL = 400 V
  R = 8 ohm
  X = 6 ohm
  connection = wye
  assumes: balanced load and supply; R >= 0
```

What the worked example means: This calculation determines the electrical characteristics of a balanced three-phase load connected in a wye configuration. Using a line-to-line voltage of 400 V and per-phase impedance defined by resistance 8 ohm and reactance 6 ohm, it computes the phase voltage as 230.94 V, phase current as 23.094 A, and line current as 23.094 A. The resulting total real power is 12800 W, reactive power is 9600 W, apparent power is 16000 W, and the power factor is 0.8. These values indicate how much useful work is performed versus how much energy oscillates between source and load.

Complex, real, reactive and apparent power from voltage and current phasors — `complex_power`

S = V I* for rms phasors: real power P, reactive power Q, apparent power |S|, the power factor and whether it lags or leads.

$$S = VI^{*} = P + jQ, \quad \mathrm{pf} = \frac{P}{|S|}$$

symbol		meaning	SI unit
`V`	in	rms voltage magnitude	V
`V_phase`	in	voltage phase angle in degrees	—
`I`	in	rms current magnitude	A
`I_phase`	in	current phase angle in degrees	—
`P`	out	real (active) power in W	W
`Q`	out	reactive power in var (positive: inductive)	W
`S`	out	apparent power in VA	W
`power_factor`	out	power factor P/|S|	—
`nature`	out	lagging (inductive), leading (capacitive) or unity	—

Applies when: rms phasors; sinusoidal steady state.

```
$ electrical-engineering solve complex_power "V=230 V" V_phase=0 "I=10 A" I_phase=-30
P = 1991.858429 W
Q = 1150 W
S = 2300 W
power_factor = 0.8660254038
nature = lagging
  by: Complex, real, reactive and apparent power from voltage and current phasors  (S = V I*
(complex conjugate of the current))
  V = 230 V
  V_phase = 0
  I = 10 A
  I_phase = -30
  assumes: rms phasors; sinusoidal steady state
```

What the worked example means: This calculation determines the total electrical power flow using voltage and current phasors. Multiplying the voltage phasor by the conjugate of the current phasor gives the complex power 2300 W. The real component represents active power 1991.86 W, which performs useful work, while the imaginary component indicates reactive power 1150 W, associated with energy storage in magnetic or electric fields. The ratio of active to apparent power defines the power factor 0.866025, indicating how efficiently the circuit utilizes the supplied energy.

Energy stored in an inductor — `inductor_energy`

The magnetic energy L I^2/2 and the flux linkage L I of an inductor carrying current I.

$$W = \tfrac{1}{2} L I^{2}$$

symbol		meaning	SI unit
`L`	in	inductance	H
`I`	in	current	A
`energy`	out	stored energy	J
`flux_linkage`	out	flux linkage L I in weber-turns	—

Applies when: a linear inductor (no saturation).

```
$ electrical-engineering solve inductor_energy "L=0.05 H" "I=2 A"
energy = 0.1 J
flux_linkage = 0.1
  by: Energy stored in an inductor  (W = L I^2 / 2)
  L = 0.05 H
  I = 2 A
  assumes: a linear inductor (no saturation)
```

What the worked example means: This calculation determines the magnetic energy stored in an inductor and its flux linkage based on the given inductance of 0.05 H and current of 2 A. The resulting energy of 0.1 J represents the work done to establish the magnetic field, while the flux linkage of 0.1 indicates the total magnetic flux coupled with the coil turns. These values are critical for analyzing circuit transients and power conversion efficiency.

First-order RC filter: cutoff, gain and phase — `rc_filter`

The cutoff frequency 1/(2 pi R C) of an RC low-pass or high-pass filter and its gain (ratio and dB) and phase at a frequency.

$$f_c = \frac{1}{2\pi RC}, \quad H_{LP} = \frac{1}{1 + jf/f_c}$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`C`	in	capacitance	F
`f`	in	signal frequency	Hz
`filter`	in	first-order low-pass or high-pass	—
`fc`	out	cutoff (-3 dB) frequency	Hz
`gain`	out	|Vout/Vin| at f	—
`gain_db`	out	gain in decibels	—
`phase_deg`	out	phase of Vout relative to Vin, degrees	—

Applies when: unloaded output; R, C, f > 0.

```
$ electrical-engineering solve rc_filter "R=1000 ohm" "C=1e-07 F" "f=2000 Hz" filter=lowpass
fc = 1591.549431 Hz
gain = 0.6226769923
gain_db = -4.11474362
phase_deg = -51.48811275
  by: First-order RC filter: cutoff, gain and phase  (H = 1/(1 + j f/fc) (low-pass) or (j f/fc)/(1
+ j f/fc) (high-pass))
  R = 1000 ohm
  C = 1e-07 F
  f = 2000 Hz
  filter = lowpass
  assumes: unloaded output; R, C, f > 0
```

What the worked example means: This calculation determines the cutoff frequency, magnitude gain, decibel gain, and phase shift for a first-order RC filter. Using resistance 1000 ohm, capacitance 1e-07 F, and signal frequency 2000 Hz, it finds that the cutoff frequency is 1591.55 Hz. The resulting voltage gain ratio is 0.622677, which corresponds to -4.11474 in decibels, with a phase shift of -51.4881 degrees between output and input.

Ideal transformer: voltages, currents and reflected resistance — `transformer`

Secondary voltage and current, primary current and the load resistance seen from the primary for an ideal transformer with a resistive load.

$$\frac{V_s}{V_p} = \frac{N_s}{N_p}, \quad R_{in} = R_L \left(\frac{N_p}{N_s} \right)^2$$

symbol		meaning	SI unit
`Np`	in	primary turns	—
`Ns`	in	secondary turns	—
`Vp`	in	primary (rms) voltage	V
`RL`	in	load resistance on the secondary	ohm
`Vs`	out	secondary voltage	V
`Is`	out	secondary (load) current	A
`Ip`	out	primary current	A
`Rin`	out	load resistance reflected to the primary	ohm
`turns_ratio`	out	Np/Ns	—

Applies when: ideal transformer (no losses, no magnetizing current).

```
$ electrical-engineering solve transformer Np=500 Ns=50 "Vp=230 V" "RL=10 ohm"
Vs = 23 V
Is = 2.3 A
Ip = 0.23 A
Rin = 1000 ohm
turns_ratio = 10
  by: Ideal transformer: voltages, currents and reflected resistance  (Vs = Vp Ns/Np, Ip = Is
Ns/Np, Rin = RL (Np/Ns)^2)
  Np = 500
  Ns = 50
  Vp = 230 V
  RL = 10 ohm
  assumes: ideal transformer (no losses, no magnetizing current)
```

What the worked example means: This calculation determines the electrical behavior of an ideal transformer based on its turns ratio and connected load. Given a primary voltage of 230 V and a load resistance of 10 ohm, the system computes the resulting secondary voltage as 23 V and the secondary current as 2.3 A. The primary current required to supply this load is found to be 0.23 A, while the resistance seen from the primary side, known as the reflected resistance, is calculated as 1000 ohm. These values illustrate how the transformer scales voltage and impedance between its windings without energy loss.

Impedance of a parallel RLC circuit — `parallel_rlc_impedance`

The complex impedance of R, L and C in parallel at a frequency: magnitude, phase angle and the two reactances.

$$\frac{1}{Z} = \frac{1}{R} + j\left(\omega C - \frac{1}{\omega L}\right)$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`L`	in	inductance	H
`C`	in	capacitance	F
`f`	in	frequency of the source in Hz	Hz
`impedance`	out	magnitude of the impedance |Z|	ohm
`phase_deg`	out	phase angle of Z in degrees (positive: inductive)	—
`real`	out	real part of Z	ohm
`imag`	out	imaginary part of Z (net reactance)	ohm
`XL`	out	inductive reactance wL	ohm
`XC`	out	capacitive reactance 1/(wC)	ohm

Applies when: R, L, C, f > 0; sinusoidal steady state.

```
$ electrical-engineering solve parallel_rlc_impedance "R=1000 ohm" "L=0.01 H" "C=1e-06 F" "f=1000
Hz"
impedance = 103.2622772 ohm
phase_deg = 84.07294187
real = 10.66309789 ohm
imag = 102.7102538 ohm
XL = 62.83185307 ohm
XC = 159.1549431 ohm
  by: Impedance of a parallel RLC circuit  (Y = 1/R + j(wC - 1/(wL)), Z = 1/Y with w = 2 pi f)
  R = 1000 ohm
  L = 0.01 H
  C = 1e-06 F
  f = 1000 Hz
  assumes: R, L, C, f > 0; sinusoidal steady state
```

What the worked example means: This calculation determines the total complex impedance for a resistor, inductor, and capacitor connected in parallel. Combining the reciprocal of resistance with the susceptances of the reactive components shows that the real part of the impedance is 10.6631 ohm and the imaginary net reactance is 102.71 ohm. The resulting magnitude of 103.262 ohm indicates how much the circuit opposes alternating current flow, while the phase angle of 84.0729 shows whether the current leads or lags the voltage.

Impedance of a series RLC circuit — `series_rlc_impedance`

The complex impedance of R, L and C in series at a frequency: magnitude, phase angle and the two reactances.

$$Z = R + j\left(\omega L - \frac{1}{\omega C}\right)$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`L`	in	inductance	H
`C`	in	capacitance	F
`f`	in	frequency of the source in Hz	Hz
`impedance`	out	magnitude of the impedance |Z|	ohm
`phase_deg`	out	phase angle of Z in degrees (positive: inductive)	—
`real`	out	real part of Z	ohm
`imag`	out	imaginary part of Z (net reactance)	ohm
`XL`	out	inductive reactance wL	ohm
`XC`	out	capacitive reactance 1/(wC)	ohm

Applies when: R >= 0, L > 0, C > 0, f > 0; sinusoidal steady state.

```
$ electrical-engineering solve series_rlc_impedance "R=100 ohm" "L=0.05 H" "C=1e-05 F" "f=1000 Hz"
impedance = 314.5621512 ohm
phase_deg = 71.46389381
real = 100 ohm
imag = 298.243771 ohm
XL = 314.1592654 ohm
XC = 15.91549431 ohm
  by: Impedance of a series RLC circuit  (Z = R + j(wL - 1/(wC)) with w = 2 pi f)
  R = 100 ohm
  L = 0.05 H
  C = 1e-05 F
  f = 1000 Hz
  assumes: R >= 0, L > 0, C > 0, f > 0; sinusoidal steady state
```

What the worked example means: This calculation determines the total opposition to alternating current in a series circuit containing resistance, inductance, and capacitance. Using the source frequency 1000 Hz, it computes the inductive reactance as 314.159 ohm and the capacitive reactance as 15.9155 ohm. The resulting complex impedance has a real part of 100 ohm and an imaginary part of 298.244 ohm, yielding a total magnitude of 314.562 ohm and a phase angle of 71.4639. A positive phase angle indicates the circuit behaves inductively at this frequency.

Phasor from polar to rectangular form — `polar_to_rectangular`

The real and imaginary parts of a phasor given as a magnitude and an angle in degrees.

$$M\angle\theta = M\cos\theta + jM\sin\theta$$

symbol		meaning	SI unit
`magnitude`	in	magnitude (non-negative)	—
`angle_deg`	in	angle in degrees	—
`real`	out	real part	—
`imag`	out	imaginary part	—

Applies when: magnitude >= 0.

```
$ electrical-engineering solve polar_to_rectangular magnitude=10 angle_deg=30
real = 8.660254038
imag = 5
  by: Phasor from polar to rectangular form  (x = M cos(theta), y = M sin(theta))
  magnitude = 10
  angle_deg = 30
  assumes: magnitude >= 0
```

What the worked example means: This calculation transforms a phasor defined by its magnitude and phase angle into rectangular coordinates. Using the magnitude 10 and the angle 30, the real component is determined as 8.66025 and the imaginary component as 5. These values represent the horizontal and vertical projections of the phasor vector on the complex plane, which are essential for performing arithmetic operations like addition and subtraction on alternating current quantities.

Phasor from rectangular to polar form — `rectangular_to_polar`

The magnitude and angle (degrees, in (-180, 180]) of a phasor given as real + j imaginary.

$$x + jy = \sqrt{x^2 + y^2} \angle \operatorname{atan2}(y, x)$$

symbol		meaning	SI unit
`real`	in	real part	—
`imag`	in	imaginary part	—
`magnitude`	out	magnitude	—
`angle_deg`	out	angle in degrees	—

Applies when: not both parts zero.

```
$ electrical-engineering solve rectangular_to_polar real=3 imag=4
magnitude = 5
angle_deg = 53.13010235
  by: Phasor from rectangular to polar form  (M = hypot(x, y), theta = atan2(y, x) (the quadrant
kept))
  real = 3
  imag = 4
  assumes: not both parts zero
```

What the worked example means: This calculation converts a complex number from rectangular form to polar form. Given the real component 3 and the imaginary component 4, it determines the vector's length and direction. The resulting magnitude is 5, representing the amplitude of the signal, while the angle is 53.1301, indicating its phase relative to the reference axis.

Power-factor correction capacitor — `power_factor_correction`

The shunt capacitance (and its reactive power) that raises a lagging load's power factor to a target value.

$$C = \frac{P(\tan\phi_1 - \tan\phi_2)}{2\pi f V^2}$$

symbol		meaning	SI unit
`P`	in	real power of the load	W
`V`	in	rms supply voltage across the capacitor	V
`f`	in	supply frequency	Hz
`pf_initial`	in	present (lagging) power factor	—
`pf_target`	in	target (lagging) power factor	—
`C`	out	correction capacitance	F
`C_uF`	out	correction capacitance in microfarads	—
`Qc`	out	reactive power the capacitor supplies, in var	W
`Q_initial`	out	reactive power before correction, in var	W
`Q_final`	out	reactive power after correction, in var	W

Applies when: single-phase, sinusoidal; both power factors lagging; target > present.

```
$ electrical-engineering solve power_factor_correction "P=10000 W" "V=230 V" "f=50 Hz"
pf_initial=0.7 pf_target=0.95
C = 0.000416101396 F
C_uF = 416.101396
Qc = 6915.19956 W
Q_initial = 10202.04061 W
Q_final = 3286.841052 W
  by: Power-factor correction capacitor  (Qc = P (tan acos pf1 - tan acos pf2), C = Qc / (2 pi f
V^2))
  P = 10000 W
  V = 230 V
  f = 50 Hz
  pf_initial = 0.7
  pf_target = 0.95
  assumes: single-phase, sinusoidal; both power factors lagging; target > present
```

What the worked example means: This calculation determines the shunt capacitor value needed to improve the power factor of an inductive load. Comparing the initial lagging power factor of 0.7 with the target of 0.95 shows that the capacitor must provide 6915.2 W of reactive power. Given the real power of 10000 W and supply voltage of 230 V, the resulting capacitance is 0.000416101 F. This reduction in reactive demand helps lower line losses and improves overall system efficiency.

Resonance, Q factor and bandwidth of a parallel RLC circuit — `parallel_rlc_resonance`

The resonant frequency, quality factor, half-power bandwidth and half-power frequencies of a parallel RLC circuit.

$$f_0 = \frac{1}{2\pi\sqrt{LC}},\ Q = R\sqrt{\frac{C}{L}}$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`L`	in	inductance	H
`C`	in	capacitance	F
`f0`	out	resonant frequency	Hz
`Q`	out	quality factor	—
`bandwidth`	out	half-power bandwidth	Hz
`f_low`	out	lower half-power frequency	Hz
`f_high`	out	upper half-power frequency	Hz

Applies when: ideal R, L, C.

```
$ electrical-engineering solve parallel_rlc_resonance "R=10 ohm" "L=0.01 H" "C=1e-06 F"
f0 = 1591.549431 Hz
Q = 0.1
bandwidth = 15915.49431 Hz
f_low = 157.5944505 Hz
f_high = 16073.08876 Hz
  by: Resonance, Q factor and bandwidth of a parallel RLC circuit  (Q = R sqrt(C/L), bandwidth =
1/(2 pi R C))
  R = 10 ohm
  L = 0.01 H
  C = 1e-06 F
  assumes: ideal R, L, C
```

What the worked example means: This calculation determines the key frequency response characteristics of an ideal parallel RLC circuit based on its component values. Given a resistance of 10 ohm, an inductance of 0.01 H, and a capacitance of 1e-06 F, the system resonates at 1591.55 Hz. The sharpness of this resonance, known as the quality factor, is calculated to be 0.1, which directly dictates the half-power bandwidth of 15915.5 Hz. Consequently, the frequencies where the power drops to half are found at 157.594 Hz and 16073.1 Hz. These metrics are critical for designing filters and tuned circuits that require specific selectivity.

Resonance, Q factor and bandwidth of a series RLC circuit — `series_rlc_resonance`

The resonant frequency, quality factor, half-power bandwidth and half-power frequencies of a series RLC circuit.

$$f_0 = \frac{1}{2\pi\sqrt{LC}},\ Q = \frac{1}{R}\sqrt{\frac{L}{C}}$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`L`	in	inductance	H
`C`	in	capacitance	F
`f0`	out	resonant frequency	Hz
`Q`	out	quality factor	—
`bandwidth`	out	half-power bandwidth	Hz
`f_low`	out	lower half-power frequency	Hz
`f_high`	out	upper half-power frequency	Hz

Applies when: ideal R, L, C.

```
$ electrical-engineering solve series_rlc_resonance "R=10 ohm" "L=0.01 H" "C=1e-06 F"
f0 = 1591.549431 Hz
Q = 10
bandwidth = 159.1549431 Hz
f_low = 1513.960154 Hz
f_high = 1673.115097 Hz
  by: Resonance, Q factor and bandwidth of a series RLC circuit  (Q = (1/R) sqrt(L/C), bandwidth =
R/(2 pi L))
  R = 10 ohm
  L = 0.01 H
  C = 1e-06 F
  assumes: ideal R, L, C
```

What the worked example means: This calculation determines the key frequency response characteristics of a series RLC circuit using the input resistance 10 ohm, inductance 0.01 H, and capacitance 1e-06 F. The resonant frequency 1591.55 Hz is where the inductive and capacitive reactances cancel each other out. The quality factor 10 quantifies the sharpness of this resonance, while the half-power bandwidth 159.155 Hz defines the range of frequencies over which the circuit delivers significant power. These values are derived from ideal component assumptions and steady-state sinusoidal operation.

RMS and average value of a periodic waveform — `rms_waveform`

The rms value (with DC offset), average, AC rms and crest factor of a sine, square, triangle or sawtooth wave.

$$V_{rms} = \sqrt{\frac{1}{T}\int_0^T v^2\,dt}$$

symbol		meaning	SI unit
`shape`	in	waveform shape	—
`amplitude`	in	peak amplitude about the average	V
`offset`	in	DC offset (average value); 0 when not given (optional)	V
`rms`	out	rms value including the offset	V
`average`	out	average value	V
`ac_rms`	out	rms of the AC part	V
`crest_factor`	out	peak over rms	—

Applies when: symmetric waveform of the stated shape about its offset.

```
$ electrical-engineering solve rms_waveform "amplitude=5 V" "offset=0 V" shape=square
rms = 5 V
average = 0 V
ac_rms = 5 V
crest_factor = 1
  by: RMS and average value of a periodic waveform  (Vrms = sqrt(offset^2 + k A^2) with k = 1/2
(sine), 1 (square), 1/3 (triangle, sawtooth))
  amplitude = 5 V
  offset = 0 V
  shape = square
  assumes: symmetric waveform of the stated shape about its offset
```

What the worked example means: This calculation determines the effective voltage, DC offset, and peak stress for a periodic waveform. Given a shape with amplitude 5 V and offset 0 V, the total root mean square is 5 V, while the AC component alone is 5 V. The average value equals the offset at 0 V. The crest factor, which indicates how spiky the signal is relative to its heating power, results in 1. This helps engineers select components that can handle both the continuous thermal load and transient voltage peaks.

Sum of sinusoids by phasor addition — `phasor_sum`

The amplitude and phase of the sum of sinusoids of one frequency, A_k cos(wt + phi_k), by adding their phasors.

$$\sum_k A_k \angle \phi_k = \sum_k A_k e^{j\phi_k}$$

symbol		meaning	SI unit
`amplitudes`	in	amplitudes (non-negative)	—
`phases_deg`	in	phase angles in degrees	—
`amplitude`	out	amplitude of the sum	—
`phase_deg`	out	phase of the sum in degrees	—
`real`	out	real part of the resultant phasor	—
`imag`	out	imaginary part of the resultant phasor	—

Applies when: one phase per amplitude; one common frequency.

```
$ electrical-engineering solve phasor_sum "amplitudes=[3, 4, 2]" "phases_deg=[0, 90, 180]"
amplitude = 4.123105626
phase_deg = 75.96375653
real = 1
imag = 4
  by: Sum of sinusoids by phasor addition  (complex addition of the phasors A_k e^{j phi_k})
  assumes: one phase per amplitude; one common frequency
```

What the worked example means: This calculation determines the single sinusoid that results from adding multiple sinusoids of the same frequency. Converting each input amplitude [3, 4, 2] and phase angle [0, 90, 180] into complex phasors, summing them, and converting back gives the final amplitude 4.12311 and phase 75.9638. The real part of this resultant vector is 1 and the imaginary part is 4, representing the combined effect in the time domain.

Control systems (11)

Closed-loop poles with unity negative feedback — `closed_loop_poles`

The poles of K*G(s)/(1 + K*G(s)): the roots of den(s) + K*num(s), and whether the closed loop is stable.

$$1 + K\,G(s) = 0 \iff \mathrm{den}(s) + K\,\mathrm{num}(s) = 0$$

symbol		meaning	SI unit
`num`	in	numerator coefficients of the open-loop G(s), highest power of s first	—
`den`	in	denominator coefficients of G(s), highest power of s first	—
`K`	in	loop gain multiplying G(s) (default 1) (optional)	—
`poles`	out	closed-loop poles (real first by real part, conjugate pairs together)	—
`max_real_part`	out	largest real part of the poles (negative when stable)	—
`stable`	out	true when every pole has a negative real part	—
`characteristic`	out	closed-loop characteristic polynomial den + K num	—

Applies when: a proper G(s); unity negative feedback.

```
$ electrical-engineering solve closed_loop_poles "num=[2]" "den=[1, 3, 0]"
poles = [-2.0,-1.0000000000000002]
max_real_part = -1
stable = true
characteristic = [1.0,3.0,2.0]
  by: Closed-loop poles with unity negative feedback  (roots of den(s) + K num(s) by the Aberth-
Ehrlich iteration)
  assumes: a proper G(s); unity negative feedback
```

What the worked example means: This calculation determines the poles of a unity negative feedback control system by finding the roots of the characteristic polynomial formed by adding the open-loop denominator to the product of the gain and the open-loop numerator. The resulting pole locations, listed as [-2, -1], define the dynamic behavior of the system. The stability is assessed by examining the real parts of these poles; specifically, if the maximum real part -1 is negative, the system is considered stable. The characteristic polynomial itself is given by [1, 3, 2].

Controllability and observability of a single-input single-output system — `controllability_observability`

The ranks of the controllability matrix [B AB ... A^(n-1)B] and the observability matrix [C; CA; ...; CA^(n-1)], and whether the system is controllable and observable.

$$\mathcal{C} = [B\ AB\ \cdots\ A^{n-1}B],\ \mathcal{O} = [C;\ CA;\ \cdots;\ CA^{n-1}]$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	—
`B`	in	input vector	—
`C`	in	output vector	—
`controllability_rank`	out	rank of the controllability matrix	—
`observability_rank`	out	rank of the observability matrix	—
`controllable`	out	full controllability rank	—
`observable`	out	full observability rank	—

Applies when: small integer or well-scaled entries (rank decided at a relative tolerance of 1e-9).

```
$ electrical-engineering solve controllability_observability "A=[[0, 1], [-2, -3]]" "B=[0, 1]" "C=
[1, 0]"
controllability_rank = 2
observability_rank = 2
controllable = true
observable = true
  by: Controllability and observability of a single-input single-output system  (Kalman rank tests
by Gaussian elimination with partial pivoting)
  assumes: small integer or well-scaled entries (rank decided at a relative tolerance of 1e-9)
```

What the worked example means: This calculation determines the structural properties of a linear dynamic system by constructing the controllability and observability matrices from the state matrix A, input vector B, and output vector C. The rank of the controllability matrix, found to be 2, indicates how many independent states can be influenced by the input. Similarly, the rank of the observability matrix, which is 2, shows how many independent states can be inferred from the output. Based on these ranks compared to the system dimension, the system is classified as yes and yes, which are essential conditions for designing full-state feedback controllers and state observers.

Frequency response (Bode magnitude and phase) at one frequency — `bode_point`

|G(jw)| as a ratio and in decibels, and the phase of G(jw), for a transfer function at an angular frequency w.

$$G(j\omega) = \frac{N(j\omega)}{D(j\omega)}, \quad 20\log_{10}|G(j\omega)|$$

symbol		meaning	SI unit
`num`	in	numerator coefficients of the open-loop G(s), highest power of s first	—
`den`	in	denominator coefficients of G(s), highest power of s first	—
`w`	in	angular frequency in rad/s	rad/s
`magnitude`	out	|G(jw)|	—
`magnitude_db`	out	20 log10 |G(jw)| in decibels	—
`phase_deg`	out	phase of G(jw) in degrees, in (-180, 180]	—
`real`	out	real part of G(jw)	—
`imag`	out	imaginary part of G(jw)	—

Applies when: no pole of G on the imaginary axis at w; w > 0.

```
$ electrical-engineering solve bode_point "num=[10]" "den=[1, 1]" "w=1 rad/s"
magnitude = 7.071067812
magnitude_db = 16.98970004
phase_deg = -45
real = 5
imag = -5
  by: Frequency response (Bode magnitude and phase) at one frequency  (Horner evaluation of
numerator and denominator at s = jw)
  w = 1 rad/s
  assumes: no pole of G on the imaginary axis at w; w > 0
```

What the worked example means: This calculation determines the frequency response of a system at the specific angular frequency 1 rad/s rad/s. Substituting this value into the transfer function defined by numerator coefficients [10] and denominator coefficients [1, 1] gives the complex gain. The resulting magnitude is 7.07107, which corresponds to 16.9897 on a decibel scale. The phase shift introduced by the system is -45 degrees, while the real and imaginary components are 5 and -5 respectively.

Gain margin and phase-crossover frequency — `gain_margin`

The gain margin of a unity-feedback loop with open-loop G(s): 1/|G(jw)| at the frequency where the phase is -180 degrees.

$$\angle G(j\omega_{pc}) = -180^\circ, \quad \mathrm{GM} = \frac{1}{|G(j\omega_{pc})|}$$

symbol		meaning	SI unit
`num`	in	numerator coefficients of the open-loop G(s), highest power of s first	—
`den`	in	denominator coefficients of G(s), highest power of s first	—
`gain_margin`	out	gain margin as a ratio	—
`gain_margin_db`	out	gain margin in decibels	—
`phase_crossover`	out	phase-crossover frequency (phase -180 degrees)	rad/s

Applies when: a proper G(s); exactly one phase crossover with G negative real.

```
$ electrical-engineering solve gain_margin "num=[10]" "den=[1, 6, 5, 0]"
gain_margin = 3
gain_margin_db = 9.542425094
phase_crossover = 2.236067977 rad/s
  by: Gain margin and phase-crossover frequency  (the phase crossover as the positive real root of
Im(N(jw) conj D(jw)) with Re G < 0 (Aberth iteration, Newton-polished))
  assumes: a proper G(s); exactly one phase crossover with G negative real
```

What the worked example means: This calculation determines the gain margin and phase crossover frequency for a unity feedback system with open-loop transfer function G(s). The phase crossover frequency, 2.23607 rad/s, is the positive real root where the imaginary part of the numerator times the conjugate of the denominator equals zero and the real part of G is negative. At this frequency, the phase angle is minus one hundred eighty degrees. The gain margin, expressed as 3 or 9.54243, represents the factor by which the open-loop gain can be increased before the closed-loop system becomes unstable.

Maximum phase lead of a lead compensator — `lead_compensator`

For C(s) = (s + z)/(s + p) with p > z: the maximum phase lead, the frequency where it occurs, and the gain there.

$$\phi_{max} = \sin^{-1}\frac{p-z}{p+z}, \; \omega_m = \sqrt{zp}$$

symbol		meaning	SI unit
`z`	in	zero (corner) frequency	rad/s

symbol		meaning	SI unit
`p`	in	pole (corner) frequency, p > z	rad/s
`phase_max_deg`	out	maximum phase lead in degrees	—
`w_max`	out	frequency of maximum phase, sqrt(z p)	rad/s
`gain_at_w_max_db`	out	gain of (s+z)/(s+p) there, in dB	—

Applies when: p > z > 0.

```
$ electrical-engineering solve lead_compensator "z=1 rad/s" "p=10 rad/s"
phase_max_deg = 54.90319877
w_max = 3.16227766 rad/s
gain_at_w_max_db = -10
  by: Maximum phase lead of a lead compensator  (phi_max = asin((p - z)/(p + z)) at w = sqrt(z p))
  z = 1 rad/s
  p = 10 rad/s
  assumes: p > z > 0
```

What the worked example means: This calculation determines the peak phase contribution of a lead compensator defined by a zero at 1 rad/s and a pole at 10 rad/s. The maximum phase lead occurs at the geometric mean frequency 3.16228 rad/s, yielding a phase shift of 54.9032. At this specific frequency, the magnitude of the transfer function is -10, which reflects the attenuation introduced by the network.

Phase margin and gain-crossover frequency — `phase_margin`

The phase margin of a unity-feedback loop with open-loop G(s): 180 degrees plus the phase of G at the frequency where |G(jw)| = 1.

$$|G(j\omega_c)| = 1, \quad \mathrm{PM} = 180^\circ + \angle G(j\omega_c)$$

symbol		meaning	SI unit
`num`	in	numerator coefficients of the open-loop G(s), highest power of s first	—
`den`	in	denominator coefficients of G(s), highest power of s first	—
`phase_margin`	out	phase margin in degrees	—
`gain_crossover`	out	gain-crossover frequency (|G| = 1)	rad/s

Applies when: a proper G(s); exactly one gain crossover.

```
$ electrical-engineering solve phase_margin "num=[4]" "den=[1, 2, 0]"
phase_margin = 51.82729237
gain_crossover = 1.572302756 rad/s
  by: Phase margin and gain-crossover frequency  (the gain crossover as the positive real root of
|N(jw)|^2 - |D(jw)|^2 (Aberth iteration, Newton-polished); PM = (arg G(jw_c) mod 360) - 180)
  assumes: a proper G(s); exactly one gain crossover
```

What the worked example means: This calculation determines the stability margins for a unity feedback system defined by the numerator coefficients [4] and denominator coefficients [1, 2, 0]. The gain crossover frequency, where the open-loop magnitude equals one, is found to be 1.5723 rad/s rad/s. At this specific

frequency, the phase margin is computed as 51.8273 degrees, representing the amount of additional phase lag required to bring the system to the verge of instability.

Routh-Hurwitz stability test — `routh_hurwitz`

The Routh array's first column, the number of roots in the right half plane and whether a characteristic polynomial is stable.

$$b_i = \frac{a_1\, a_{0,i+1} - a_0\, a_{1,i+1}}{a_1}$$

symbol		meaning	SI unit
`coefficients`	in	characteristic polynomial coefficients, highest power of s first	—
`rhp_roots`	out	number of roots with positive real part (sign changes in the first column)	—
`stable`	out	true when every root has a negative real part	—
`first_column`	out	the first column of the Routh array	—

Applies when: no zero in the first column of the Routh array (the special cases are not handled).

```
$ electrical-engineering solve routh_hurwitz "coefficients=[1, 2, 3, 4, 5]"
rhp_roots = 2
stable = false
first_column = [1.0,2.0,1.0,-6.0,5.0]
  by: Routh-Hurwitz stability test  (Routh array built row by row; sign changes of its first column
counted)
  assumes: no zero in the first column of the Routh array (the special cases are not handled)
```

What the worked example means: This calculation constructs the Routh array from the characteristic polynomial coefficients provided as input. Examining the signs of the elements in the first column gives the number of sign changes. The result shows that the first column contains values [1, 2, 1, -6, 5]. There are 2 sign changes, which indicates the number of poles with positive real parts. Consequently, the system is determined to be stable if no is true.

Steady-state error of a unity-feedback system — `steady_state_error`

The system type, the error constant (Kp, Kv or Ka) and the steady-state error for a unit step, ramp or parabola input.

$$e_{ss} = \lim_{s \to 0} \frac{s\, R(s)}{1 + G(s)}$$

symbol		meaning	SI unit
`num`	in	numerator coefficients of the open-loop G(s), highest power of s first	—
`den`	in	denominator coefficients of G(s), highest power of s first	—
`input`	in	the test input: unit step 1/s, unit ramp 1/s^2 or unit parabola t^2/2 (1/s^3)	—
`steady_state_error`	out	steady-state error (or 'infinite')	—
`error_constant`	out	position (Kp), velocity (Kv) or acceleration (Ka) error constant	—

symbol		meaning	SI unit
`system_type`	out	number of open-loop poles at the origin	—

Applies when: a stable closed loop; unity negative feedback; unit-amplitude test input.

```
$ electrical-engineering solve steady_state_error "num=[10]" "den=[1, 3, 2]" input=step
steady_state_error = 0.1666666667
error_constant = 5
system_type = 0
  by: Steady-state error of a unity-feedback system  (system type from the poles at the origin;
error constant lim s^k G(s); e_ss = 1/(1+Kp) for a step, 1/Kv for a ramp, 1/Ka for a parabola)
  input = step
  assumes: a stable closed loop; unity negative feedback; unit-amplitude test input
```

In the worked example, num = [10], den = [1, 3, 2], input = step go through the equation above and give steady_state_error = 0.1666666667, error_constant = 5, system_type = 0. It applies only when a stable closed loop; unity negative feedback; unit-amplitude test input: outside that it says what is wrong instead of returning a number.

Steady-state speed and current of a permanent-magnet DC motor — `dc_motor`

Speed, current, powers and efficiency of a DC motor with armature resistance R, torque constant Kt, back-emf constant Ke, viscous friction b and load torque T at supply voltage V.

$$\omega = \frac{K_t V - RT}{K_t K_e + Rb}$$

symbol		meaning	SI unit
`V`	in	armature supply voltage	V
`R`	in	armature resistance	ohm
`Kt`	in	torque constant in N*m/A	—
`Ke`	in	back-emf constant in V*s/rad	—
`T`	in	load torque	N*m
`b`	in	viscous friction in N*m*s/rad	—
`speed`	out	steady-state speed	rad/s
`rpm`	out	speed in revolutions per minute	—
`current`	out	armature current	A
`input_power`	out	electrical input power	W
`output_power`	out	mechanical output power to the load	W
`efficiency`	out	output over input power	—

Applies when: steady state; linear model (no saturation, constant parameters).

```
$ electrical-engineering solve dc_motor "V=12 V" "R=1 ohm" Kt=0.05 Ke=0.05 "T=0.02 N*m" b=0.0001
speed = 223.0769231 rad/s
rpm = 2130.2277
current = 0.8461538462 A
input_power = 10.15384615 W
output_power = 4.461538462 W
efficiency = 0.4393939394
  by: Steady-state speed and current of a permanent-magnet DC motor  (solve V = R I + Ke w and Kt I
= b w + T)
  V = 12 V
  R = 1 ohm
  Kt = 0.05
  Ke = 0.05
  T = 0.02 N*m
  b = 0.0001
  assumes: steady state; linear model (no saturation, constant parameters)
```

What the worked example means: This calculation determines the operating point of a permanent-magnet DC motor by solving the coupled electrical and mechanical equations. Given the supply voltage, resistance, torque constant, back-emf constant, friction, and load torque, the model computes the resulting angular velocity in radians per second and revolutions per minute, alongside the drawn current. The final speed of 223.077 rad/s and current of 0.846154 A indicate how the motor balances electromagnetic torque against mechanical losses and external load at equilibrium.

Step-response specifications of a standard second-order system — `second_order_step`

Percent overshoot, peak time, 10-90% rise time and exact 2% settling time of wn^2/(s^2 + 2 zeta wn s + wn^2) for a unit step.

$$M_p = e^{-\zeta\pi/\sqrt{1-\zeta^2}}, \quad t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}$$

symbol		meaning	SI unit
`zeta`	in	damping ratio (0 < zeta < 1)	—
`wn`	in	undamped natural frequency	rad/s
`percent_overshoot`	out	maximum overshoot in percent	—
`overshoot`	out	maximum overshoot as a fraction of the final value	—
`peak_time`	out	time of the first peak	s
`rise_time`	out	10% to 90% rise time	s
`t10`	out	time to reach 10% of the final value	s
`t90`	out	time to reach 90% of the final value	s
`settling_time`	out	exact 2% settling time (last time the response leaves the band)	s
`damped_frequency`	out	damped natural frequency wn sqrt(1 - zeta^2)	rad/s

Applies when: 0 < zeta < 1 (underdamped); unit step from rest; unity DC gain.

```
$ electrical-engineering solve second_order_step zeta=0.5 "wn=4 rad/s"
percent_overshoot = 16.30335348
overshoot = 0.1630335348
peak_time = 0.9068996821 s
rise_time = 0.4093932368 s
t10 = 0.122057324 s
t90 = 0.5314505608 s
settling_time = 2.019087243 s
damped_frequency = 3.464101615 rad/s
  by: Step-response specifications of a standard second-order system  (closed-form overshoot and
peak time; rise and settling times by bisection on the exact response (the last 2% crossing lies
between the last peak above the band and the following zero of the error))
  zeta = 0.5
  wn = 4 rad/s
  assumes: 0 < zeta < 1 (underdamped); unit step from rest; unity DC gain
```

What the worked example means: The system response to a unit step input is characterized by its transient performance metrics. With a damping ratio of 0.5 and an undamped natural frequency of 4 rad/s, the maximum overshoot reaches 0.163034 of the final value, corresponding to 16.3034 percent. The first peak occurs at 0.9069 s, while the rise time from ten to ninety percent is 0.409393 s. The exact two percent settling time is calculated as 2.01909 s, indicating when the response permanently stays within the error band.

Transfer function of a single-input single-output state-space model — `state_space_to_tf`

G(s) = C (sI - A)^-1 B + D as numerator and denominator coefficients for x' = Ax + Bu, y = Cx + Du.

$$G(s) = C(sI - A)^{-1}B + D$$

symbol		meaning	SI unit
`A`	in	state matrix (n x n)	—
`B`	in	input vector (n entries)	—
`C`	in	output vector (n entries)	—
`D`	in	feedthrough	—
`num`	out	numerator coefficients (highest power first, length n + 1)	—
`den`	out	denominator coefficients det(sI - A), monic	—
`order`	out	system order n	—

Applies when: a square A with B and C of matching size; coefficients below 1e-12 of the largest are written as 0.

```
$ electrical-engineering solve state_space_to_tf "B=[0, 1]" "C=[1, 0]" "A=[[0, 1], [-2, -3]]" D=0
num = [0.0,0.0,1.0]
den = [1.0,3.0,2.0]
order = 2
  by: Transfer function of a single-input single-output state-space model  (Faddeev-LeVerrier
recursion for det(sI - A) and adj(sI - A))
  D = 0
  assumes: a square A with B and C of matching size; coefficients below 1e-12 of the largest are
written as 0
```

What the worked example means: This calculation derives the single-input single-output transfer function from the state-space matrices A, B, C, and D. The resulting numerator coefficients are [0, 0, 1] and the denominator coefficients are [1, 3, 2], which define the poles of the system. This representation allows for frequency-domain analysis of the dynamics described by the state matrix A with dimension 2.

DC circuits (18)

Current divider — `current_divider`

How a current I splits between two resistors in parallel.

$$I_1 = I\frac{R_2}{R_1 + R_2}$$

symbol		meaning	SI unit
`I`	in	total current into the parallel pair	A
`R1`	in	first branch resistor	ohm
`R2`	in	second branch resistor	ohm
`I1`	out	current through R1	A
`I2`	out	current through R2	A

Applies when: R1, R2 > 0.

```
$ electrical-engineering solve current_divider "I=6 A" "R1=4 ohm" "R2=2 ohm"
I1 = 2 A
I2 = 4 A
  by: Current divider  (I1 = I R2/(R1 + R2))
  I = 6 A
  R1 = 4 ohm
  R2 = 2 ohm
  assumes: R1, R2 > 0
```

What the worked example means: This calculation determines how the total input current of 6 A divides between two parallel resistors. The branch containing the first resistor with a value of 4 ohm carries 2 A, and the branch with the second resistor valued at 2 ohm carries 4 A. These results show that higher resistance in a specific branch leads to lower current flow through that path, consistent with the principles of parallel circuits.

Delta-to-wye (Pi-to-T) resistor transformation — `delta_to_wye`

The three wye (star) resistances equivalent to a delta (triangle) of resistances between terminals a, b, c.

$$R_a = \frac{R_{ab}R_{ca}}{R_{ab} + R_{bc} + R_{ca}}$$

symbol		meaning	SI unit
`Rab`	in	delta resistance between terminals a and b	ohm
`Rbc`	in	delta resistance between terminals b and c	ohm
`Rca`	in	delta resistance between terminals c and a	ohm

symbol		meaning	SI unit
`Ra`	out	wye arm at terminal a	ohm
`Rb`	out	wye arm at terminal b	ohm
`Rc`	out	wye arm at terminal c	ohm

Applies when: every resistance positive.

```
$ electrical-engineering solve delta_to_wye "Rab=10 ohm" "Rbc=20 ohm" "Rca=30 ohm"
Ra = 5 ohm
Rb = 3.333333333 ohm
Rc = 10 ohm
  by: Delta-to-wye (Pi-to-T) resistor transformation  (each arm is the product of the two adjacent
delta resistances over their sum)
  Rab = 10 ohm
  Rbc = 20 ohm
  Rca = 30 ohm
  assumes: every resistance positive
```

What the worked example means: This calculation converts a delta network with resistances 10 ohm, 20 ohm, and 30 ohm into an equivalent star network. The resulting wye arms are 5 ohm at terminal a, 3.33333 ohm at terminal b, and 10 ohm at terminal c. Each wye resistance represents the equivalent value seen from its respective terminal to the common neutral point when viewed against the other two terminals.

Equivalent capacitance of capacitors in parallel — `parallel_capacitance`

The single capacitance equivalent to two or more capacitors connected in parallel.

$$C_{eq} = \sum_{i} C_i$$

symbol		meaning	SI unit
`C`	in	the capacitance values, all in one unit (F, mF, uF, nF ...; a list, e.g. '10, 22, 47')	—
`unit`	in	the unit the values are written in (the result is in the same unit) (optional)	—
`equivalent`	out	equivalent capacitance, in the unit of the inputs	—
`count`	out	number of capacitors	—
`unit`	out	the unit of the result	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve parallel_capacitance "C=[3, 6]" unit=uF
equivalent = 9
count = 2
unit = uF
  by: Equivalent capacitance of capacitors in parallel  (the sum of the values)
  unit = uF
  assumes: every value positive; at least two values
```

What the worked example means: This calculation determines the single equivalent capacitance for two or more capacitors connected in parallel. Adding the individual capacitance values together gives a total

capacitance that represents the combined effect of all components sharing common terminals. The result, 9, is expressed in the same unit as the input values, ensuring consistency across the calculation.

Equivalent capacitance of capacitors in series — `series_capacitance`

The single capacitance equivalent to two or more capacitors connected in series.

$$\frac{1}{C_{eq}} = \sum_i \frac{1}{C_i}$$

symbol		meaning	SI unit
`C`	in	the capacitance values, all in one unit (F, mF, uF, nF ...; a list, e.g. '10, 22, 47')	—
`unit`	in	the unit the values are written in (the result is in the same unit) (optional)	—
`equivalent`	out	equivalent capacitance, in the unit of the inputs	—
`count`	out	number of capacitors	—
`unit`	out	the unit of the result	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve series_capacitance "C=[3, 6]" unit=uF
equivalent = 2
count = 2
unit = uF
  by: Equivalent capacitance of capacitors in series  (the reciprocal of the sum of reciprocals)
  unit = uF
  assumes: every value positive; at least two values
```

What the worked example means: This calculation determines the single equivalent capacitance for capacitors connected in a series configuration. Taking the reciprocal of the sum of the reciprocals of each individual capacitance value shows that the total effective capacitance is 2. This result indicates that the combined storage capacity is lower than that of any individual capacitor in the chain, which is essential for designing timing circuits and filters where reduced charge storage is required.

Equivalent inductance of inductors in parallel — `parallel_inductance`

The single inductance equivalent to two or more inductors connected in parallel.

$$\frac{1}{L_{eq}} = \sum_i \frac{1}{L_i}$$

symbol		meaning	SI unit
`L`	in	the inductance values, all in one unit (H, mH, uH, nH ...; a list, e.g. '10, 22, 47')	—
`unit`	in	the unit the values are written in (the result is in the same unit) (optional)	—
`equivalent`	out	equivalent inductance, in the unit of the inputs	—
`count`	out	number of inductors	—
`unit`	out	the unit of the result	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve parallel_inductance "L=[10, 40]" unit=mH
equivalent = 8
count = 2
unit = mH
  by: Equivalent inductance of inductors in parallel  (the reciprocal of the sum of reciprocals)
  unit = mH
  assumes: every value positive; at least two values
```

In the worked example, L = [10, 40], unit = mH go through the equation above and give equivalent = 8, count = 2, unit = mH. It applies only when every value positive; at least two values: outside that it says what is wrong instead of returning a number.

Equivalent inductance of inductors in series — `series_inductance`

The single inductance equivalent to two or more inductors connected in series.

$$L_{eq} = \sum_{i} L_i$$

symbol		meaning	SI unit
`L`	in	the inductance values, all in one unit (H, mH, uH, nH ...; a list, e.g. '10, 22, 47')	—
`unit`	in	the unit the values are written in (the result is in the same unit) (optional)	—
`equivalent`	out	equivalent inductance, in the unit of the inputs	—
`count`	out	number of inductors	—
`unit`	out	the unit of the result	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve series_inductance "L=[10, 22]" unit=mH
equivalent = 32
count = 2
unit = mH
  by: Equivalent inductance of inductors in series  (the sum of the values)
  unit = mH
  assumes: every value positive; at least two values
```

In the worked example, L = [10, 22], unit = mH go through the equation above and give equivalent = 32, count = 2, unit = mH. It applies only when every value positive; at least two values: outside that it says what is wrong instead of returning a number.

Equivalent resistance of a series-parallel resistor network — `resistor_network`

The equivalent resistance of a network written with + for series and || for parallel, e.g. 10 + (20 || 30).

$$R_s = R_1 + R_2, \quad R_p = \left(\frac{1}{R_1} + \frac{1}{R_2} \right)^{-1}$$

symbol		meaning	SI unit
`network`	in	the network in ohms: + joins in series, || joins in parallel ('||' binds tighter than '+'), parentheses group	—
`equivalent`	out	equivalent resistance	ohm
`resistors`	out	number of resistors in the network	—

Applies when: every resistance positive; at least two resistors; '||' binds tighter than '+'.

```
$ electrical-engineering solve resistor_network "network=10 + (20 || 30)"
equivalent = 22 ohm
resistors = 3
  by: Equivalent resistance of a series-parallel resistor network  (recursive reduction of the
expression tree: series groups add, parallel groups add reciprocally)
  network = 10 + (20 || 30)
  assumes: every resistance positive; at least two resistors; '||' binds tighter than '+'
```

What the worked example means: This calculation determines the total resistance seen by a source connected to the resistor network defined as 10 + (20 || 30). By recursively combining series elements through addition and parallel elements through reciprocal addition, the complex arrangement reduces to a single value. The result, 22 ohm, represents the effective opposition to current flow for the entire circuit, while the count of individual components involved in this reduction is 3.

Equivalent resistance of resistors in parallel — `parallel_resistance`

The single resistance equivalent to two or more resistors connected in parallel.

$$\frac{1}{R_{eq}} = \sum_{i} \frac{1}{R_i}$$

symbol		meaning	SI unit
`R`	in	the resistance values in ohms (a list, e.g. '10, 20, 30')	—
`equivalent`	out	equivalent resistance	ohm
`count`	out	number of resistors	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve parallel_resistance "R=[10, 40]"
equivalent = 8 ohm
count = 2
  by: Equivalent resistance of resistors in parallel  (the reciprocal of the sum of reciprocals)
  assumes: every value positive; at least two values
```

What the worked example means: This calculation determines the single resistance value that replaces multiple resistors connected in parallel. By applying the reciprocal of the sum of reciprocals method to the input values, the model computes an equivalent resistance of 8 ohm. This result represents the total opposition to current flow across the shared nodes, which is always lower than the smallest individual resistor in the group.

Equivalent resistance of resistors in series — `series_resistance`

The single resistance equivalent to two or more resistors connected in series.

$$R_{eq} = \sum_i R_i$$

symbol		meaning	SI unit
`R`	in	the resistance values in ohms (a list, e.g. '10, 20, 30')	—
`equivalent`	out	equivalent resistance	ohm
`count`	out	number of resistors	—

Applies when: every value positive; at least two values.

```
$ electrical-engineering solve series_resistance "R=[10, 20, 30]"
equivalent = 60 ohm
count = 3
  by: Equivalent resistance of resistors in series  (the sum of the values)
  assumes: every value positive; at least two values
```

What the worked example means: This calculation determines the single resistance value that replaces a series of resistors. It sums the individual resistance values provided in [10, 20, 30] to produce the total equivalent resistance, reported as 60 ohm. This result represents the total opposition to current flow in a circuit where components are connected end-to-end.

Maximum power transfer — `max_power_transfer`

The load resistance that draws the most power from a source with Thevenin voltage Vth and resistance Rth, and that power.

$$P_{max} = \frac{V_{th}^2}{4R_{th}}$$

symbol		meaning	SI unit
`Vth`	in	Thevenin (open-circuit) voltage	V
`Rth`	in	Thevenin resistance	ohm
`RL`	out	load resistance for maximum power (= Rth)	ohm
`Pmax`	out	maximum power delivered to the load	W
`efficiency`	out	fraction of the source power reaching the load at that point	—

Applies when: a resistive load; Rth > 0.

```
$ electrical-engineering solve max_power_transfer "Vth=12 V" "Rth=6 ohm"
RL = 6 ohm
Pmax = 6 W
efficiency = 0.5
  by: Maximum power transfer  (RL = Rth, Pmax = Vth^2/(4 Rth))
  Vth = 12 V
  Rth = 6 ohm
  assumes: a resistive load; Rth > 0
```

What the worked example means: This calculation determines the optimal load resistance for extracting the greatest possible power from a source defined by its Thevenin voltage and resistance. By setting the load resistance equal to the Thevenin resistance, the system achieves maximum power transfer, resulting in a power output of 6 W at a load value of 6 ohm. At this specific operating point, the efficiency of power transfer is fixed at 0.5, meaning half of the total power generated is dissipated within the source itself rather than delivered to the load.

Mesh currents from the mesh equations (Kirchhoff's voltage law) — `mesh_currents`

Solve the mesh equations R i = v of a circuit: the resistance matrix and the source-voltage vector give every mesh current.

$$\mathbf{R}\,\mathbf{i} = \mathbf{v}$$

symbol		meaning	SI unit
`A`	in	mesh resistance matrix in ohms (self-resistances on the diagonal, minus the shared resistances off it)	—
`b`	in	net source voltage around each mesh in volts, in the mesh-current direction	—
`currents`	out	the mesh currents in amperes, in the order of the rows	—
`first`	out	the first mesh current	A

Applies when: square matrix; one source voltage per mesh; a non-singular matrix.

```
$ electrical-engineering solve mesh_currents "A=[[10, -4], [-4, 8]]" "b=[12, 0]"
currents = [1.5,0.7500000000000001]
first = 1.5 A
  by: Mesh currents from the mesh equations (Kirchhoff's voltage law)  (Gaussian elimination with
partial pivoting)
  assumes: square matrix; one source voltage per mesh; a non-singular matrix
```

What the worked example means: This calculation determines the individual loop currents in a planar circuit by solving the system of linear equations formed from Kirchhoff's voltage law. The resistance matrix and source voltage vector define how electrical energy drives current through each independent path. The resulting values represent the magnitude and direction of flow for every mesh, expressed as [1.5, 0.75] amperes.

Node voltages from the nodal equations (Kirchhoff's current law) — `nodal_voltages`

Solve the nodal equations G v = i of a circuit: the conductance matrix and the injected-current vector give every node voltage.

$$\mathbf{G}\,\mathbf{v} = \mathbf{i}$$

symbol		meaning	SI unit
`A`	in	nodal conductance matrix in siemens (sums of conductances on the diagonal, minus the shared conductances off it)	—
`b`	in	current injected into each node by sources, in amperes	—
`voltages`	out	the node voltages in volts, in the order of the rows	—
`first`	out	the first node voltage	V

Applies when: square matrix; one injected current per node; a non-singular matrix.

```
$ electrical-engineering solve nodal_voltages "A=[[0.5, -0.25], [-0.25, 0.75]]" "b=[2, 0]"
voltages = [4.8,1.6]
first = 4.8 V
  by: Node voltages from the nodal equations (Kirchhoff's current law)  (Gaussian elimination with partial pivoting)
  assumes: square matrix; one injected current per node; a non-singular matrix
```

What the worked example means: This calculation determines the electrical potential at each node in a circuit by solving a system of linear equations based on Kirchhoff's Current Law. The inputs consist of the nodal conductance matrix, which represents how easily current flows between nodes, and the vector of injected currents from sources. By applying Gaussian elimination with partial pivoting to these values, the method computes the steady-state voltage at every node. The resulting voltages, such as 4.8 V for the first node, indicate the electric potential relative to the reference ground, allowing for the analysis of current flow through individual branches.

Node voltages of a DC resistive circuit from its netlist — `dc_node_voltages`

Every node voltage, every voltage-source current and the power delivered, for a circuit of resistors and independent sources written as a netlist.

$$\begin{bmatrix} G & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} v \\ i \end{bmatrix} = \begin{bmatrix} i_s \\ e \end{bmatrix}$$

symbol		meaning	SI unit
`netlist`	in	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes> (flowing from the first node through the source to the second); node 0 is ground	—
`node_voltages`	out	voltage of every node relative to node 0, in volts	—
`source_currents`	out	current through each voltage source from its + node to its - node, in amperes	—
`power_delivered`	out	total power delivered by the sources (= power dissipated in the resistors)	W

Applies when: a ground node 0; every resistance positive; a unique solution (no floating node, no loop of voltage sources).

```
$ electrical-engineering solve dc_node_voltages "netlist=V1 1 0 12; R1 1 2 1000; R2 2 0 2000."
node_voltages = {"1":12.0,"2":8.0}
source_currents = {"V1":-0.004}
power_delivered = 0.048 W
  by: Node voltages of a DC resistive circuit from its netlist  (modified nodal analysis solved by
Gaussian elimination)
  netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000.
  assumes: a ground node 0; every resistance positive; a unique solution (no floating node, no loop
of voltage sources)
```

What the worked example means: This calculation determines the electrical state of a DC resistive circuit defined by its netlist. By applying modified nodal analysis, it solves for the voltage at every node relative to ground, represented as {1: 12, 2: 8}. It also computes the current flowing through each independent voltage source, given by {V1: -0.004}, and sums the energy supplied by all sources, which equals the total dissipation in the resistors, shown as 0.048 W. These values provide a complete description of how the circuit distributes potential and current under steady-state conditions.

Norton equivalent of a DC circuit between two nodes — `norton_equivalent`

The Norton current and resistance seen between two nodes of a resistive circuit with independent sources.

$$I_N = \frac{V_{th}}{R_{th}}, \quad R_N = R_{th}$$

symbol		meaning	SI unit
`netlist`	in	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes>; node 0 is ground	—
`node_a`	in	the terminal node (the + side of the equivalent)	—
`node_b`	in	the reference terminal node	—
`In`	out	Norton (short-circuit) current from node a to node b	A
`Rn`	out	Norton resistance (equal to the Thevenin resistance)	ohm

Applies when: both nodes in the circuit; no ideal voltage source directly across the terminals; a unique solution (no floating node).

```
$ electrical-engineering solve norton_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2 1000;
R2 2 0 2000."
In = 0.012 A
Rn = 666.6666667 ohm
  by: Norton equivalent of a DC circuit between two nodes  (Thevenin voltage and resistance by
modified nodal analysis, then I_N = V_th / R_th)
  node_a = 2
  node_b = 0
  netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000.
  assumes: both nodes in the circuit; no ideal voltage source directly across the terminals; a
unique solution (no floating node)
```

In the worked example, node_a = 2, node_b = 0, netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000. go through the equation above and give In = 0.012 A, Rn = 666.6666667 ohm. It applies only when both nodes in the circuit; no ideal voltage source directly across the terminals; a unique solution (no floating node): outside that it says what is wrong instead of returning a number.

Thevenin equivalent of a DC circuit between two nodes — `thevenin_equivalent`

The Thevenin voltage and resistance seen between two nodes of a resistive circuit with independent sources.

$$V_{th} = v_a - v_b\big|_{open}, \quad R_{th} = \frac{v_a - v_b}{1\,\mathrm{A}}\Big|_{sources\ off}$$

symbol		meaning	SI unit
`netlist`	in	elements separated by ';': R<name> <node> <node> <ohms>, V<name> <node+> <node-> <volts>, I<name> <node> <node> <amperes>; node 0 is ground	—
`node_a`	in	the terminal node (the + side of the equivalent)	—
`node_b`	in	the reference terminal node	—
`Vth`	out	Thevenin (open-circuit) voltage, node a relative to node b	V
`Rth`	out	Thevenin resistance	ohm

Applies when: both nodes in the circuit; no ideal voltage source directly across the terminals; a unique solution (no floating node).

```
$ electrical-engineering solve thevenin_equivalent node_a=2 node_b=0 "netlist=V1 1 0 12; R1 1 2
1000; R2 2 0 2000."
Vth = 8 V
Rth = 666.6666667 ohm
  by: Thevenin equivalent of a DC circuit between two nodes  (open-circuit voltage by modified
nodal analysis; resistance by a 1 A test current with the independent sources set to zero)
  node_b = 0
  netlist = V1 1 0 12; R1 1 2 1000; R2 2 0 2000.
  node_a = 2
  assumes: both nodes in the circuit; no ideal voltage source directly across the terminals; a
unique solution (no floating node)
```

What the worked example means: This calculation finds the Thevenin equivalent circuit seen from two specific terminals. It determines the open-circuit voltage between node a and node b, denoted as 8 V, and the equivalent resistance, 666.667 ohm, by deactivating independent sources and applying a test current. The result represents a simplified linear model of the complex network, allowing easy analysis of how any load connected to these terminals will behave.

Voltage divider — `voltage_divider`

The output voltage across R2 of two resistors in series across Vin (R1 on top, R2 to ground), and the current.

$$V_{out} = V_{in}\frac{R_2}{R_1 + R_2}$$

symbol		meaning	SI unit
`Vin`	in	input voltage	V
`R1`	in	top resistor (from the input to the output)	ohm
`R2`	in	bottom resistor (from the output to ground)	ohm
`Vout`	out	output voltage across R2 (unloaded)	V
`I`	out	current through the divider	A

Applies when: no load on the output; R1, R2 > 0.

```
$ electrical-engineering solve voltage_divider "Vin=12 V" "R1=1000 ohm" "R2=2000 ohm"
Vout = 8 V
I = 0.004 A
  by: Voltage divider  (Vout = Vin R2 / (R1 + R2))
  Vin = 12 V
  R1 = 1000 ohm
  R2 = 2000 ohm
  assumes: no load on the output; R1, R2 > 0
```

What the worked example means: This calculation determines the voltage at the node between two series resistors connected across an input source. Using the input voltage of 12 V and resistor values of 1000 ohm for the top element and 2000 ohm for the bottom element, the resulting output voltage is 8 V. The total current flowing through this path is 0.004 A. This result assumes no external load is connected to the output node, meaning the measured voltage represents the ideal potential difference across the lower resistor.

Wheatstone bridge output — `wheatstone_bridge`

The output voltage of a Wheatstone bridge (left arm R1 over R2, right arm R3 over R4, supply Vs), whether it is balanced, and the R4 that balances it.

$$V_{out} = V_s \left(\frac{R_2}{R_1 + R_2} - \frac{R_4}{R_3 + R_4} \right)$$

symbol		meaning	SI unit
`Vs`	in	bridge supply voltage	V
`R1`	in	left arm, top	ohm
`R2`	in	left arm, bottom	ohm
`R3`	in	right arm, top	ohm
`R4`	in	right arm, bottom	ohm
`Vout`	out	voltage of the left midpoint relative to the right midpoint	V
`balanced`	out	true when R1 R4 = R2 R3	—
`R4_balance`	out	the R4 that would balance the bridge	ohm

Applies when: an unloaded output (ideal voltmeter).

```
$ electrical-engineering solve wheatstone_bridge "Vs=10 V" "R1=100 ohm" "R2=200 ohm" "R3=150 ohm"
"R4=250 ohm"
Vout = 0.4166666667 V
balanced = false
R4_balance = 300 ohm
  by: Wheatstone bridge output  (difference of the two divider outputs)
  Vs = 10 V
  R1 = 100 ohm
  R2 = 200 ohm
  R3 = 150 ohm
  R4 = 250 ohm
  assumes: an unloaded output (ideal voltmeter)
```

What the worked example means: This calculation determines the voltage difference between the midpoints of a Wheatstone bridge using the supply voltage and four resistor values. The result, 0.416667 V, represents the potential at the left midpoint relative to the right midpoint. If this value is zero, the bridge is balanced, meaning the ratio of resistors in the left arm matches the ratio in the right arm. When unbalanced, the magnitude of 0.416667 V indicates the degree of deviation from equilibrium.

Wye-to-delta (T-to-Pi) resistor transformation — `wye_to_delta`

The three delta (triangle) resistances equivalent to a wye (star) of resistances at terminals a, b, c.

$$R_{ab} = \frac{R_a R_b + R_b R_c + R_c R_a}{R_c}$$

symbol		meaning	SI unit
`Ra`	in	wye arm at terminal a	ohm
`Rb`	in	wye arm at terminal b	ohm
`Rc`	in	wye arm at terminal c	ohm
`Rab`	out	delta resistance between a and b	ohm
`Rbc`	out	delta resistance between b and c	ohm
`Rca`	out	delta resistance between c and a	ohm

Applies when: every resistance positive.

```
$ electrical-engineering solve wye_to_delta "Ra=5 ohm" "Rb=10 ohm" "Rc=20 ohm"
Rab = 17.5 ohm
Rbc = 70 ohm
Rca = 35 ohm
  by: Wye-to-delta (T-to-Pi) resistor transformation  (each delta resistance is the sum of pairwise
products of the arms over the opposite arm)
  Ra = 5 ohm
  Rb = 10 ohm
  Rc = 20 ohm
  assumes: every resistance positive
```

What the worked example means: This calculation determines the equivalent delta resistances for a given star configuration. Using the wye arm values 5 ohm, 10 ohm, and 20 ohm, the method computes each line-to-line resistance by summing the pairwise products of the arms and dividing by the opposite arm. For instance,

the resistance between terminals a and b results in 17.5 ohm, representing the direct connection in the triangular topology that behaves identically to the original star arrangement at these three nodes.

Digital logic (9)

Analog-to-digital conversion: output code and resolution — `adc_conversion`

The output code (truncating converter), its binary form, the LSB size and the quantized voltage of an ideal n-bit ADC with reference Vref.

$$\text{code} = \left\lfloor \frac{V_{in}}{V_{ref}/2^{n}} \right\rfloor$$

symbol		meaning	SI unit
`Vin`	in	input voltage (0 to Vref)	V
`Vref`	in	full-scale reference voltage	V
`bits`	in	resolution in bits	—
`code`	out	output code	—
`binary`	out	code in binary	—
`lsb`	out	LSB size Vref/2^n	V
`lsb_mV`	out	LSB in millivolts	—
`quantized`	out	voltage of the code, code x LSB	V

Applies when: 0 <= Vin <= Vref; truncating (floor) quantizer; Vref itself gives the top code.

```
$ electrical-engineering solve adc_conversion "Vin=1.2 V" "Vref=5 V" bits=8
code = 61
binary = 00111101
lsb = 0.01953125 V
lsb_mV = 19.53125
quantized = 1.19140625 V
  by: Analog-to-digital conversion: output code and resolution  (code = floor(Vin / LSB), capped at
2^n - 1)
  Vin = 1.2 V
  Vref = 5 V
  assumes: 0 <= Vin <= Vref; truncating (floor) quantizer; Vref itself gives the top code
```

What the worked example means: This calculation determines the digital representation of an analog input voltage using an ideal n-bit truncating converter. Given an input voltage of 1.2 V and a reference voltage of 5 V, the least significant bit size is calculated as 0.0195312 V. The resulting output code is 61, which corresponds to a quantized voltage of 1.19141 V. This process maps the continuous input signal to discrete digital levels, illustrating the resolution limits inherent in analog-to-digital conversion.

Binary to Gray code and Gray code to binary — `gray_code`

The reflected binary (Gray) code of a binary word, or the binary word of a Gray code, with the same width.

$$g = b \oplus (b \gg 1)$$

symbol		meaning	SI unit
`value`	in	the bits	—
`direction`	in	conversion direction	—
`result`	out	the converted bits	—
`decimal`	out	the result read as an unsigned number	—

Applies when: 1 to 32 bits.

```
$ electrical-engineering solve gray_code value=1011 direction=to_gray
result = 1110
decimal = 14
  by: Binary to Gray code and Gray code to binary  (g = b XOR (b >> 1); back: b = g XOR (g >> 1)
XOR (g >> 2) ...)
  value = 1011
  direction = to_gray
  assumes: 1 to 32 bits
```

What the worked example means: This calculation converts the provided bit pattern 1011 into its corresponding reflected binary Gray code 1110. The conversion applies an exclusive OR operation between the original binary word and a version shifted right by one position. This method ensures that the resulting sequence maintains the property where consecutive numbers differ by exactly one bit, which is essential for minimizing errors in state transitions. The output 14 represents this converted bit pattern interpreted as an unsigned integer value.

Equivalence of two Boolean expressions — `boolean_equivalence`

Whether two Boolean expressions are equal for every input, with a counterexample row when they are not.

$$F_1 \equiv F_2 \iff F_1 \oplus F_2 = 0$$

symbol		meaning	SI unit
`expression1`	in	first expression (same notation as boolean_evaluate)	—
`expression2`	in	second expression	—
`equivalent`	out	true when the two agree on every row	—
`counterexample`	out	the first row where they differ ('none' if equivalent)	—

Applies when: at most 12 variables.

```
$ electrical-engineering solve boolean_equivalence "expression1=(A·B)'" "expression2=A' + B'"
equivalent = true
counterexample = none
  by: Equivalence of two Boolean expressions  (exhaustive comparison over the truth table)
  expression2 = A' + B'
  expression1 = (A·B)'
  assumes: at most 12 variables
```

What the worked example means: This calculation checks whether the two provided Boolean expressions, (A·B)' and A' + B', yield identical results for every possible combination of their variables. The method involves

an exhaustive comparison over the truth table, verifying that the exclusive OR of the two functions equals zero. The final determination is recorded as yes, which indicates if the expressions are logically equal. If they are not equivalent, the system identifies a specific input row where they differ, reported as none. This process ensures that circuit simplifications or design changes preserve the original logical behavior.

Evaluate a Boolean expression — `boolean_evaluate`

The value (0 or 1) of a Boolean expression for given values of its variables.

$$F(A, B, \ldots) \in \{0, 1\}$$

symbol		meaning	SI unit
`expression`	in	the expression: AND (&, ·, *), OR (|, +), XOR (^), NOT (~, !, NOT, or a postfix '), parentheses; NOT > AND > XOR > OR	—
`values`	in	the variables' values, e.g. 'A=1, B=0, C=1'	—
`value`	out	value of the expression (0 or 1)	—

Applies when: every variable given 0 or 1; products written with an operator (A·B, not AB).

```
$ electrical-engineering solve boolean_evaluate "expression=A AND (B OR NOT C)" "values=A=1, B=0,
C=0"
value = 1
  by: Evaluate a Boolean expression  (recursive-descent parse with NOT > AND > XOR > OR, then
evaluation)
  expression = A AND (B OR NOT C)
  values = A=1, B=0, C=0
  assumes: every variable given 0 or 1; products written with an operator (A·B, not AB)
```

What the worked example means: This calculation determines the final logical state of a Boolean expression by applying standard precedence rules, where NOT operations are processed before AND, which precedes XOR, and finally OR. The process begins with the input expression A AND (B OR NOT C) and the specific binary values assigned to its variables, provided as A=1, B=0, C=0. Each variable is substituted with its given value, and the expression is parsed recursively to resolve parentheses and operators in the correct order. The resulting single bit, either zero or one, is reported as 1, representing the definitive truth value of the entire logic circuit or statement under those specific conditions.

Minimal sum of products from minterms (with don't-cares) — `minterms_to_sop`

A minimal sum of products (fewest terms, then fewest literals) of F(A, B, C, D) = Σm(...) + d(...), the Karnaugh-map result.

$$F = \sum m(\ldots) + \sum d(\ldots)$$

symbol		meaning	SI unit
`minterms`	in	minterm numbers (A is the most significant bit)	—
`variables`	in	number of variables (1 to 4)	—
`dont_cares`	in	don't-care rows (optional)	—

symbol		meaning	SI unit
`minimal_sop`	out	a minimal sum of products in A, B, C, D (A' = NOT A)	—
`terms`	out	number of product terms	—
`literals`	out	number of literals	—
`variables`	out	the variables	—

Applies when: 1 to 4 variables; minterms and don't-cares disjoint and in range.

```
$ electrical-engineering solve minterms_to_sop "minterms=[0, 1, 2, 5, 6, 7]" variables=3
minimal_sop = A'·B' + A·C + B·C'
terms = 3
literals = 6
variables = A, B, C
  by: Minimal sum of products from minterms (with don't-cares)  (Quine-McCluskey prime implicants
over minterms and don't-cares; exact minimum cover of the minterms)
  assumes: 1 to 4 variables; minterms and don't-cares disjoint and in range
```

In the worked example, minterms = [0, 1, 2, 5, 6, 7], variables = 3 go through the equation above and give minimal_sop = A'·B' + A·C + B·C', terms = 3, literals = 6, variables = A, B, C. It applies only when 1 to 4 variables; minterms and don't-cares disjoint and in range: outside that it says what is wrong instead of returning a number.

Number base conversion (binary, octal, decimal, hexadecimal) — `base_conversion`

A whole number written in base 2, 8, 10 or 16, given in all four bases.

$$N = \sum_{i} d_i \, b^i$$

symbol		meaning	SI unit
`value`	in	the digits (0b, 0o, 0x prefixes allowed)	—
`base`	in	the base the value is written in	—
`decimal`	out	value in base 10	—
`binary`	out	base 2	—
`octal`	out	base 8	—
`hexadecimal`	out	base 16 (upper-case digits)	—

Applies when: a non-negative whole number below 2^53; digits valid for the base.

```
$ electrical-engineering solve base_conversion value=1011 base=2
decimal = 11
binary = 1011
octal = 13
hexadecimal = B
  by: Number base conversion (binary, octal, decimal, hexadecimal)  (positional value, then
repeated division by the target base)
  value = 1011
  base = 2
  assumes: a non-negative whole number below 2^53; digits valid for the base
```

What the worked example means: This calculation determines the equivalent representation of a non-negative integer across four common numerical bases. Starting with the input value 1011 defined in base 2, the algorithm first computes its positional decimal value using the sum of digits multiplied by powers of the base. This decimal result 11 is then converted into binary, octal, and hexadecimal formats through repeated division by the target bases. The final outputs provide the same magnitude expressed as 1011, 13, and B, allowing engineers to view data at different levels of hardware abstraction.

Truth table and minimal sum of products of a Boolean expression — `truth_table_sop`

The minterms (truth table), the canonical sum of products and a minimal sum of products (fewest terms, then fewest literals) of an expression in up to 4 variables.

$$F = \sum m(\dots)$$

symbol		meaning	SI unit
`expression`	in	the expression: AND (&, ·, *), OR (|, +), XOR (^), NOT (~, !, NOT, or a postfix '), parentheses; NOT > AND > XOR > OR	—
`variables`	out	the variables in truth-table order (first = most significant bit)	—
`minterms`	out	rows where the expression is 1	—
`maxterms`	out	rows where it is 0	—
`canonical_sop`	out	sum of all minterms	—
`minimal_sop`	out	a minimal sum of products (A' means NOT A, · means AND)	—
`terms`	out	number of product terms in the minimal form	—
`literals`	out	number of literals in the minimal form	—

Applies when: 1 to 4 variables.

```
$ electrical-engineering solve truth_table_sop "expression=A·B + A·B' + A'·B"
variables = A, B
minterms = [1.0,2.0,3.0]
maxterms = [0.0]
canonical_sop = A'·B + A·B' + A·B
minimal_sop = A + B
terms = 2
literals = 2
  by: Truth table and minimal sum of products of a Boolean expression  (truth table; Quine-
McCluskey prime implicants; essential primes, then an exact search for the smallest cover)
  expression = A·B + A·B' + A'·B
  assumes: 1 to 4 variables
```

What the worked example means: This calculation determines the simplest logical form for the input expression A·B + A·B' + A'·B. It first generates a truth table to identify all rows where the output is one, listed as [1, 2, 3]. Using these values, it constructs the canonical sum of products and then applies the Quine-McCluskey algorithm to find prime implicants. The result is a minimized expression A + B that uses the fewest possible product terms 2 and literals 2, providing an efficient implementation for digital logic design.

Two's complement representation of a signed integer — `twos_complement_encode`

The n-bit two's complement bit pattern (and its hexadecimal form) of a signed whole number.

$$x_{2c} = x \bmod 2^n$$

symbol		meaning	SI unit
`value`	in	the signed whole number	—
`bits`	in	word width in bits	—
`pattern`	out	the bit pattern, most significant bit first	—
`hexadecimal`	out	the pattern in hexadecimal	—
`unsigned`	out	the pattern read as an unsigned number	—

Applies when: -2^(n-1) <= value < 2^(n-1).

```
$ electrical-engineering solve twos_complement_encode value=-5 bits=8
pattern = 11111011
hexadecimal = FB
unsigned = 251
  by: Two's complement representation of a signed integer  (value modulo 2^n (for a negative
number: invert the bits of |value| and add 1))
  assumes: -2^(n-1) <= value < 2^(n-1)
```

What the worked example means: This calculation determines the fixed-width binary representation of a signed integer using the two's complement system. Given the input value -5 and a word width of 8 bits, the resulting bit pattern is 11111011. When this specific sequence of bits is interpreted as an unsigned number, it yields the value 251. This method ensures that positive and negative integers share the same arithmetic rules at the hardware level, allowing subtraction to be performed via addition of the negated operand.

Value of a two's complement bit pattern — `twos_complement_decode`

The signed value of an n-bit two's complement pattern (n = the number of digits written).

$$x = -b_{n-1}2^{n-1} + \sum_{i<n-1} b_i 2^i$$

symbol		meaning	SI unit
`pattern`	in	the bits, most significant first	—
`value`	out	signed value	—
`bits`	out	word width	—

Applies when: only 0 and 1; at most 64 bits.

```
$ electrical-engineering solve twos_complement_decode pattern=11111011
value = -5
bits = 8
  by: Value of a two's complement bit pattern  (unsigned value minus 2^n when the sign bit is 1)
  pattern = 11111011
  assumes: only 0 and 1; at most 64 bits
```

What the worked example means: This calculation determines the signed integer represented by the binary sequence 11111011 using two's complement notation. The method interprets the most significant bit as the sign indicator; if it is one, the unsigned magnitude is reduced by twice the weight of that bit position to yield the negative result. The final output, -5, reflects this signed interpretation over a word width of 8 bits.

Diodes and transistors (4)

DC operating point of a voltage-divider biased BJT — `bjt_divider_bias`

Base, collector and emitter currents and the node voltages of an NPN transistor with divider bias (R1 to VCC, R2 to ground), collector resistor RC and emitter resistor RE.

$$I_B = \frac{V_{TH} - V_{BE}}{R_{TH} + (\beta + 1)R_E}, \quad V_{TH} = V_{CC}\frac{R_2}{R_1 + R_2}$$

symbol		meaning	SI unit
`VCC`	in	supply voltage	V
`R1`	in	divider resistor from VCC to the base	ohm
`R2`	in	divider resistor from the base to ground	ohm
`RC`	in	collector resistor	ohm
`RE`	in	emitter resistor	ohm
`beta`	in	DC current gain	—
`VBE`	in	base-emitter voltage in conduction (e.g. 0.7 V)	V
`IC`	out	collector current	A
`IC_mA`	out	collector current in milliamperes	—
`IB`	out	base current	A
`IE`	out	emitter current	A

symbol		meaning	SI unit
`VB`	out	base voltage	V
`VE`	out	emitter voltage	V
`VC`	out	collector voltage	V
`VCE`	out	collector-emitter voltage	V

Applies when: NPN in the forward-active region (VCE >= 0.2 V); constant VBE and beta.

```
$ electrical-engineering solve bjt_divider_bias "VCC=12 V" "VBE=0.7 V" "R1=10000 ohm" "R2=2200 ohm"
"RC=3600 ohm" "RE=1000 ohm" beta=100
IC = 0.001424015309 A
IC_mA = 1.424015309
IB = 1.424015309e-05 A
IE = 0.001438255462 A
VB = 2.138255462 V
VE = 1.438255462 V
VC = 6.873544889 V
VCE = 5.435289428 V
  by: DC operating point of a voltage-divider biased BJT  (Thevenin equivalent of the divider, then
IB = (VTH - VBE)/(RTH + (beta+1) RE) (the exact analysis, base current included))
  VCC = 12 V
  VBE = 0.7 V
  R1 = 10000 ohm
  R2 = 2200 ohm
  RC = 3600 ohm
  RE = 1000 ohm
  beta = 100
  assumes: NPN in the forward-active region (VCE >= 0.2 V); constant VBE and beta
```

What the worked example means: This analysis determines the steady-state voltages and currents for an NPN transistor using voltage-divider bias. Finding the Thevenin equivalent of the resistor network gives the base current as 1.42402e-05 A, which sets the collector current to 0.00142402 A and the emitter current to 0.00143826 A. The resulting node voltages are base at 2.13826 V, emitter at 1.43826 V, and collector at 6.87354 V, confirming the device operates in the forward-active region with a collector-emitter voltage of 5.43529 V.

MOSFET drain current in saturation (square law) — `mosfet_saturation`

Drain current, overdrive voltage and transconductance of an n-channel MOSFET in saturation.

$$I_D = \tfrac{1}{2} k_n' \frac{W}{L} (V_{GS} - V_{TH})^2$$

symbol		meaning	SI unit
`kn`	in	process transconductance parameter mu_n Cox in A/V^2	—
`WL`	in	aspect ratio W/L	—
`VGS`	in	gate-source voltage	V
`VTH`	in	threshold voltage	V
`VDS`	in	drain-source voltage	V

symbol		meaning	SI unit
`ID`	out	drain current	A
`ID_mA`	out	drain current in milliamperes	—
`VOV`	out	overdrive VGS - VTH	V
`gm`	out	transconductance in A/V (siemens)	—

Applies when: saturation: VGS > VTH and VDS >= VGS - VTH; no channel-length modulation.

```
$ electrical-engineering solve mosfet_saturation kn=0.0002 WL=10 "VGS=2 V" "VTH=0.7 V" "VDS=5 V"
ID = 0.00169 A
ID_mA = 1.69
VOV = 1.3 V
gm = 0.0026
  by: MOSFET drain current in saturation (square law)  (ID = (1/2) kn (W/L) (VGS - VTH)^2)
  kn = 0.0002
  WL = 10
  VGS = 2 V
  VTH = 0.7 V
  VDS = 5 V
  assumes: saturation: VGS > VTH and VDS >= VGS - VTH; no channel-length modulation
```

What the worked example means: This calculation determines the operating point of an n-channel MOSFET in saturation. Using the process parameter 0.0002 and aspect ratio 10, along with gate-source voltage 2 V and threshold voltage 0.7 V, the overdrive voltage is found to be 1.3 V. Consequently, the drain current is 0.00169 A (1.69), and the small-signal transconductance is 0.0026. These values indicate how strongly the transistor conducts and responds to input changes under these bias conditions.

Operating point of a diode in series with a resistor (Shockley equation) — `diode_circuit`

The current and the diode voltage of a forward-biased diode I = Is (exp(VD/(n VT)) - 1) in series with a resistor across a DC source.

$$V_s = IR + nV_T \ln\left(1 + \frac{I}{I_s}\right)$$

symbol		meaning	SI unit
`Vs`	in	source voltage (forward)	V
`R`	in	series resistor	ohm
`Is`	in	saturation current	A
`n`	in	ideality factor	—
`VT`	in	thermal voltage kT/q (about 25.85 mV at 300 K)	V
`I`	out	circuit current	A
`I_mA`	out	current in milliamperes	—
`VD`	out	diode voltage	V
`VR`	out	resistor voltage	V

Applies when: forward bias (Vs > 0); the Shockley model without series resistance or breakdown.

```
$ electrical-engineering solve diode_circuit "Vs=5 V" "R=1000 ohm" "Is=1e-14 A" n=1 "VT=0.02585 V"
I = 0.004307509625 A
I_mA = 4.307509625
VD = 0.6924903752 V
VR = 4.307509625 V
  by: Operating point of a diode in series with a resistor (Shockley equation)  (bisection then
Newton on Vs = I R + n VT ln(1 + I/Is))
  Vs = 5 V
  R = 1000 ohm
  Is = 1e-14 A
  n = 1
  VT = 0.02585 V
  assumes: forward bias (Vs > 0); the Shockley model without series resistance or breakdown
```

What the worked example means: This calculation determines the steady-state operating point for a forward-biased diode in series with a resistor. By solving the nonlinear circuit equation, it finds that the current through the branch is 4.30751 when driven by a source voltage of 5 V. The resulting voltage across the diode is 0.69249 V, while the remaining potential difference of 4.30751 V appears across the series resistance. This result reflects the balance between the exponential current-voltage characteristic of the semiconductor junction and the linear ohmic behavior of the resistor.

Zener diode shunt regulator — `zener_regulator`

Load voltage and the currents of a Zener shunt regulator (series resistor R from the input, Zener and load in parallel).

$$I_Z = \frac{V_{in} - V_Z}{R} - \frac{V_Z}{R_L}$$

symbol		meaning	SI unit
`Vin`	in	input voltage	V
`R`	in	series resistor	ohm
`Vz`	in	Zener (breakdown) voltage	V
`RL`	in	load resistance	ohm
`VL`	out	load voltage	V
`IR`	out	current in the series resistor	A
`IL`	out	load current	A
`IZ`	out	Zener current	A
`IZ_mA`	out	Zener current in milliamperes	—
`PZ`	out	power dissipated in the Zener	W

Applies when: the Zener is in breakdown (ideal: constant Vz); positive values.

```
$ electrical-engineering solve zener_regulator "Vin=20 V" "R=1000 ohm" "Vz=12 V" "RL=2000 ohm"
VL = 12 V
IR = 0.008 A
IL = 0.006 A
IZ = 0.002 A
IZ_mA = 2
PZ = 0.024 W
  by: Zener diode shunt regulator  (I_R = (Vin - Vz)/R, I_L = Vz/RL, I_Z = I_R - I_L)
  Vin = 20 V
  R = 1000 ohm
  Vz = 12 V
  RL = 2000 ohm
  assumes: the Zener is in breakdown (ideal: constant Vz); positive values
```

What the worked example means: This calculation determines the operating point of a Zener diode shunt regulator. The series resistor current 0.008 A A is derived from the difference between the input voltage 20 V and the Zener breakdown voltage 12 V, divided by the resistance 1000 ohm. The load current 0.006 A A flows through the load resistance 2000 ohm, resulting in a stable load voltage 12 V. The remaining current, 0.002 A A or 2 mA, passes through the Zener diode, dissipating power of 0.024 W W to maintain regulation.

Operational amplifiers (5)

Difference amplifier output (four-resistor op-amp circuit) — `difference_amplifier`

The output of the classic difference amplifier: V1 through R1 to the inverting input, R2 from output to that input, V2 through R3 to the non-inverting input, R4 from it to ground.

$$V_{out} = \frac{R_4}{R_3 + R_4}\left(1 + \frac{R_2}{R_1}\right)V_2 - \frac{R_2}{R_1}V_1$$

symbol		meaning	SI unit
`V1`	in	voltage at the inverting side	V
`V2`	in	voltage at the non-inverting side	V
`R1`	in	input resistor to the inverting input	ohm
`R2`	in	feedback resistor	ohm
`R3`	in	input resistor to the non-inverting input	ohm
`R4`	in	resistor from the non-inverting input to ground	ohm
`Vout`	out	output voltage	V
`gain_inverting`	out	gain from V1, -R2/R1	—
`gain_noninverting`	out	gain from V2, R4/(R3+R4) (1 + R2/R1)	—

Applies when: ideal op-amp (infinite gain, no saturation).

```
$ electrical-engineering solve difference_amplifier "R1=10000 ohm" "R2=100000 ohm" "R3=10000 ohm"
"R4=100000 ohm" "V1=1 V" "V2=1.5 V"
Vout = 5 V
gain_inverting = -10
gain_noninverting = 10
  by: Difference amplifier output (four-resistor op-amp circuit)  (superposition with v+ = v-)
  R1 = 10000 ohm
  R2 = 100000 ohm
  R3 = 10000 ohm
  R4 = 100000 ohm
  V1 = 1 V
  V2 = 1.5 V
  assumes: ideal op-amp (infinite gain, no saturation)
```

What the worked example means: This calculation determines the output voltage of a four-resistor difference amplifier using superposition. The result, 5 V, combines the scaled input from the inverting side and the non-inverting side. The effective amplification for the first signal is -10, while the second signal sees a gain of 10. These values indicate how strongly each input influences the final output relative to their respective source voltages.

Inverting amplifier gain and output — `inverting_amplifier`

The voltage gain -Rf/Rin of an ideal inverting amplifier and its output for an input voltage.

$$A_v = -\frac{R_f}{R_{in}}$$

symbol		meaning	SI unit
`Rin`	in	input resistor	ohm
`Rf`	in	feedback resistor	ohm
`Vin`	in	input voltage	V
`gain`	out	closed-loop voltage gain	—
`Vout`	out	output voltage	V

Applies when: ideal op-amp, no saturation.

```
$ electrical-engineering solve inverting_amplifier "Rin=10000 ohm" "Rf=47000 ohm" "Vin=0.5 V"
gain = -4.7
Vout = -2.35 V
  by: Inverting amplifier gain and output  (gain = -Rf/Rin)
  Rin = 10000 ohm
  Rf = 47000 ohm
  Vin = 0.5 V
  assumes: ideal op-amp, no saturation
```

What the worked example means: This calculation determines the closed-loop voltage gain and the resulting output voltage for an ideal inverting operational amplifier. The gain is derived from the ratio of the feedback resistor to the input resistor, specifically negative Rf divided by Rin. Using the provided input voltage, the final output is computed as the product of this gain and the input signal. For example, with an input resistor of 10000 ohm, a feedback resistor of 47000 ohm, and an input voltage of 0.5 V, the system

yields a gain of -4.7 and an output voltage of -2.35 V. This configuration effectively scales and inverts the input signal.

Inverting summing amplifier output — `summing_amplifier`

The output of an ideal inverting summing amplifier: Vout = -Rf (V1/R1 + V2/R2 + ...).

$$V_{out} = -R_f \sum_k \frac{V_k}{R_k}$$

symbol		meaning	SI unit
`V`	in	input voltages in volts	—
`R`	in	input resistors in ohms (one per input)	—
`Rf`	in	feedback resistor	ohm
`Vout`	out	output voltage	V
`gains`	out	gain of each input, -Rf/R_k	—

Applies when: ideal op-amp (infinite gain, no saturation); non-inverting input grounded.

```
$ electrical-engineering solve summing_amplifier "Rf=100000 ohm" "V=[1, 2, -0.5]" "R=[10000, 20000,
10000]"
Vout = -15 V
gains = [-10.0,-5.0,-10.0]
  by: Inverting summing amplifier output  (virtual ground at the inverting input; Kirchhoff's
current law there)
  Rf = 100000 ohm
  assumes: ideal op-amp (infinite gain, no saturation); non-inverting input grounded
```

What the worked example means: This calculation determines the output voltage of an ideal inverting summing amplifier using the feedback resistor and individual input resistors. The result, -15 V, represents the negative sum of each input voltage divided by its corresponding resistance, scaled by the feedback resistor value. This configuration effectively performs analog addition where each signal contributes according to its specific gain factor defined by the ratio of the feedback resistor to the input resistor.

Non-inverting amplifier gain and output — `noninverting_amplifier`

The voltage gain 1 + Rf/Rg of an ideal non-inverting amplifier and its output.

$$A_v = 1 + \frac{R_f}{R_g}$$

symbol		meaning	SI unit
`Rg`	in	resistor from the inverting input to ground	ohm
`Rf`	in	feedback resistor	ohm
`Vin`	in	input voltage (at the non-inverting input)	V
`gain`	out	closed-loop voltage gain	—
`Vout`	out	output voltage	V

Applies when: ideal op-amp, no saturation.

```
$ electrical-engineering solve noninverting_amplifier "Rg=10000 ohm" "Rf=90000 ohm" "Vin=0.2 V"
gain = 10
Vout = 2 V
  by: Non-inverting amplifier gain and output  (gain = 1 + Rf/Rg)
  Rg = 10000 ohm
  Rf = 90000 ohm
  Vin = 0.2 V
  assumes: ideal op-amp, no saturation
```

What the worked example means: This calculation determines the closed-loop voltage gain and the final output voltage for an ideal non-inverting amplifier. By using the feedback resistor value of 90000 ohm and the ground resistor value of 10000 ohm, the system computes a gain of 10. When this gain is applied to the input signal of 0.2 V, the predicted output voltage becomes 2 V. This result indicates how much the input signal is amplified without any phase inversion.

Op-amp integrator output for a constant input — `opamp_integrator`

The output of an ideal inverting integrator (R in, C feedback) after a constant input Vin has been applied for time t, from an initial output V0.

$$V_{out}(t) = V_0 - \frac{1}{RC}\int_0^t V_{in}\,dt$$

symbol		meaning	SI unit
`R`	in	input resistor	ohm
`C`	in	feedback capacitor	F
`Vin`	in	constant input voltage	V
`t`	in	integration time	s
`V0`	in	output voltage at t = 0	V
`Vout`	out	output voltage at t	V
`slope`	out	output slope dVout/dt in V/s	—

Applies when: ideal op-amp, no saturation; constant input.

```
$ electrical-engineering solve opamp_integrator "R=10000 ohm" "C=1e-06 F" "Vin=0.5 V" "t=0.005 s"
"V0=0 V"
Vout = -0.25 V
slope = -50
  by: Op-amp integrator output for a constant input  (Vout = V0 - Vin t/(RC))
  R = 10000 ohm
  C = 1e-06 F
  Vin = 0.5 V
  t = 0.005 s
  V0 = 0 V
  assumes: ideal op-amp, no saturation; constant input
```

What the worked example means: This calculation determines the final output voltage of an ideal inverting integrator after a constant input has been applied for a specific duration. Starting from an initial voltage, the circuit accumulates the input signal over time, producing a linear ramp at the output. The computed result shows that with an input resistor and feedback capacitor creating a specific time constant, the output reaches -0.25 V while changing at a rate of -50. This reflects how the capacitor charges or discharges in response to the steady input current generated by the voltage across the input resistor.

Transients (3)

First-order (RC or RL) transient response — `first_order_transient`

The value at time t of any voltage in a first-order circuit going from its initial value V0 toward its final value Vf with time constant tau.

$$v(t) = V_f + (V_0 - V_f)e^{-t/\tau}$$

symbol		meaning	SI unit
`V0`	in	initial value (at t = 0+)	V
`Vf`	in	final (steady-state) value	V
`tau`	in	time constant (RC or L/R)	s
`t`	in	time after the switching	s
`v`	out	value at time t	V
`fraction_complete`	out	fraction of the change completed, 1 - exp(-t/tau)	—

Applies when: a single time constant; a step change at t = 0.

```
$ electrical-engineering solve first_order_transient "V0=0 V" "Vf=10 V" "tau=0.001 s" "t=0.002 s"
v = 8.646647168 V
fraction_complete = 0.8646647168
  by: First-order (RC or RL) transient response  (v(t) = Vf + (V0 - Vf) exp(-t/tau))
  V0 = 0 V
  Vf = 10 V
  tau = 0.001 s
  t = 0.002 s
  assumes: a single time constant; a step change at t = 0
```

What the worked example means: This calculation determines the instantaneous voltage in a first-order circuit during its transient response. Using the initial value 0 V, the final steady-state value 10 V, and the time constant 0.001 s, it computes the voltage at the specific time 0.002 s. The result, 8.64665 V, indicates the potential difference across the component at that moment. Additionally, the fraction of the transition completed is 0.864665, showing how close the system is to reaching its final state relative to the start point.

Step response of a series RLC circuit — `series_rlc_step`

Capacitor voltage and loop current at time t after a DC step V is applied to a series RLC circuit at rest, with the damping classification.

$$LC\,\ddot{v}_C + RC\,\dot{v}_C + v_C = V, \quad \alpha = \frac{R}{2L},\ \omega_0 = \frac{1}{\sqrt{LC}}$$

symbol		meaning	SI unit
`R`	in	resistance	ohm
`L`	in	inductance	H
`C`	in	capacitance	F
`V`	in	step voltage applied at t = 0	V
`t`	in	time after the step	s
`vC`	out	capacitor voltage at t	V
`i`	out	loop current at t	A
`i_mA`	out	loop current in milliamperes	—
`alpha`	out	neper frequency R/(2L) in 1/s	—
`omega0`	out	undamped resonant frequency 1/sqrt(LC)	rad/s
`zeta`	out	damping ratio alpha/omega0	—
`response`	out	underdamped, critically damped or overdamped	—

Applies when: circuit at rest before the step (vC = 0, i = 0); ideal components.

```
$ electrical-engineering solve series_rlc_step "R=10 ohm" "L=0.1 H" "C=0.0001 F" "V=12 V" "t=0.005
s"
vC = 10.41435345 V
i = 0.2992853965 A
i_mA = 299.2853965
alpha = 50
omega0 = 316.227766 rad/s
zeta = 0.158113883
response = underdamped
  by: Step response of a series RLC circuit  (closed-form solution of L C vC'' + R C vC' + vC = V
by the roots of the characteristic equation)
  R = 10 ohm
  L = 0.1 H
  C = 0.0001 F
  V = 12 V
  t = 0.005 s
  assumes: circuit at rest before the step (vC = 0, i = 0); ideal components
```

What the worked example means: This calculation determines the transient behavior of a series RLC circuit after a DC step voltage is applied. Using the component values, it computes the neper frequency as 50 per second and the undamped resonant frequency as 316.228 rad/s radians per second. The resulting damping ratio is 0.158114. Based on these parameters, the system exhibits an underdamped response. At time 0.005 s, the capacitor voltage reaches 10.4144 V volts and the loop current is 0.299285 A amperes, which corresponds to 299.285 milliamperes.

Time for a first-order response to reach a value — `first_order_time_to_value`

How long a first-order (RC or RL) response from V0 toward Vf with time constant tau takes to reach a value v.

$$t = \tau \ln \frac{V_f - V_0}{V_f - v}$$

symbol		meaning	SI unit
`V0`	in	initial value	V
`Vf`	in	final value	V
`tau`	in	time constant	s
`v`	in	value to reach	V
`t`	out	time to reach v	s
`time_constants`	out	the same time in time constants	—

Applies when: v between V0 and Vf.

```
$ electrical-engineering solve first_order_time_to_value "V0=0 V" "Vf=10 V" "tau=0.002 s" "v=5 V"
t = 0.001386294361 s
time_constants = 0.6931471806
  by: Time for a first-order response to reach a value  (t = -tau ln(1 - (v - V0)/(Vf - V0)))
  V0 = 0 V
  Vf = 10 V
  tau = 0.002 s
  v = 5 V
  assumes: v between V0 and Vf
```

What the worked example means: This calculation finds the elapsed time for a first-order response starting at 0 V and ending at 10 V with a time constant of 0.002 s to reach the target value 5 V. The result is 0.00138629 s, meaning the circuit takes this duration to achieve that specific level. Expressed in normalized units, this corresponds to 0.693147 time constants, indicating how far along the exponential curve the target lies.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.